SZABIST University, Karachi Campus

Faculty of Computing and Engineering Sciences

Software Requirements Specification

for

SecureFlow

Program Analysis and Docker-Based Dynamic Verification of Broken Access Control and Unsafe Input Handling in Flask Applications

Version 1.0 (draft for advisor review)

Prepared by

Syed Munsif (2312347)

Maheen Arshad (2312404)

Advisor

Muhammad Ahsan

Final Year Project (FYP)

October 2026

Advisor review (required before submission)	
Reviewed by	Muhammad Ahsan
Comments	
Signature	
Date	

Table of Contents

Revision History **4**

1. Introduction **1**

1.1 Purpose 1

1.2 Document Conventions 1

1.3 Intended Audience and Reading Suggestions 2

1.4 Product Scope 3

1.5 References 4

2. Overall Description **5**

2.1 Product Perspective 5

2.2 Product Functions 6–7

2.3 User Classes and Characteristics 8

2.4 Operating Environment 8

2.5 Design and Implementation Constraints 9

2.6 User Documentation 9

2.7 Assumptions and Dependencies 10

3. External Interface Requirements **11**

3.1 User Interfaces 11–15

3.2 Hardware Interfaces 16

3.3 Software Interfaces 16

3.4 Communications Interfaces 17

4. System Features **18**

4.1 Run Security Scan 18

4.2 Complete First-Run Setup 22

4.3 Configure Test Setup 25

4.4 Prepare the Test Environment 29

4.5 Detect Suspected Vulnerabilities 32

4.6 Verify Suspects with Live Tests 35

4.7 View Scan Results 38

4.8 View Finding Details and Proof 41

4.9 Apply a Suggested Fix and Re-test 43

4.10 Export Report 46

4.11 Handle Scan Errors and Fallbacks 48

4.12 View Scan History 51

5. Other Nonfunctional Requirements **54**

5.1 Performance Requirements 54

5.2 Safety Requirements 55

5.3 Security Requirements 56

5.4 Software Quality Attributes.....57
5.5 Business Rules.....58
6. Other Requirements.....59
Appendix A: Glossary.....60–61
Appendix B: Analysis Models.....62
B.1 Use Case Diagram.....62
B.2 Scan Activity Flow.....63
B.3 Finding State Diagram.....64
B.4 Entity Relationship Diagram.....65
B.5 Data Dictionary.....66–72
Appendix C: To Be Determined List.....73

Revision History

Name	Date	Reason For Changes	Version
Syed Munsif, Maheen Arshad	18-Oct-2026	First complete draft for advisor review	0.1
Syed Munsif, Maheen Arshad	21-Oct-2026	Added FYP-1 and FYP-2 feature allocation; corrected page breaks and table of contents	0.2
Syed Munsif, Maheen Arshad	22-Oct-2026	Added local database, View Scan History (4.12) and the ERD	0.3
Syed Munsif, Maheen Arshad	TBD	Changes after advisor review	1.0

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for SecureFlow, version 1.0. SecureFlow is a security testing tool for Python Flask web applications. It runs inside Visual Studio Code and finds two kinds of security problems: Broken Access Control and Unsafe Input Handling. It does not stop at reading code. After finding a suspected problem, it tests the suspicion against a disposable copy of the developer's app and reports the problem only when the test proves it.

This SRS covers the whole of SecureFlow version 1.0: the VS Code extension, the analysis engine and the Docker-based test environment. It states what the system must do and the limits within which it must work. It does not describe how the system will be built; that is left to the design document.

1.2 Document Conventions

The document follows the SRS template for the course (based on the Wiegers template) and the department's formatting rules: Times New Roman, 12 pt body text, bold headings, 1.2 line spacing, normal margins and justified paragraphs. Main section titles are 16 pt bold, sub-section titles 14 pt bold and third-level titles 12 pt bold.

- The words "shall" and "must" mark a mandatory requirement. The word "should" marks a recommendation.
- Every functional requirement has its own identifier made of a feature tag and a number, for example SCAN-4, and its own priority. Priorities are not inherited.
- Priority is High (the product cannot work or be demonstrated without it), Medium (important, but a workaround exists) or Low (useful, can be dropped if time is short).
- Status is "Pending" for every requirement because no part of the system has been built yet.
- Section 4 describes each feature as a use case in the department's tabular format, followed by a table of requirements.
- "TBD" marks information that is not yet decided. All TBD items are collected in Appendix C.

1.3 Intended Audience and Reading Suggestions

The document is written for the following readers.

- **The project advisor and evaluators** check that the requirements are complete, clear and achievable. They should read Sections 1 and 2, then Section 4, then Section 5.
- **The developers (the two team members)** use the document as the basis for design and implementation. They should read Sections 2 to 5 in full and use Appendix B for the models.
- **Testers** write test cases from the use cases and requirements in Section 4 and the quality targets in Section 5.
- **Future users and maintainers** can read Sections 1 and 2 and the glossary in Appendix A to understand what the tool does and what it does not do.

The rest of the document is organised as follows. Section 2 describes the product in general. Section 3 describes the interfaces. Section 4 lists the system features as use cases with their requirements. Section 5 gives the non-functional requirements. Section 6 lists other requirements. The appendices contain the glossary, the analysis models and the list of open items.

1.4 Product Scope

Many students and small teams build Flask web applications without a security background. Common tools such as Bandit read code and report long lists of possible problems, and many of them are false alarms. Developers then either ignore the list or lose time checking each item by hand.

SecureFlow aims to give a short, trustworthy list. It combines two steps: program analysis finds suspected problems in the code, and a Docker sandbox then attacks a safe copy of the app to confirm each suspicion. A problem is reported as confirmed only when the attack actually works. The developer sees the proof (the request that was sent and the response that came back) and can apply a ready-made fix and re-run the same test.

Objectives of version 1.0:

- Detect and confirm Broken Access Control problems: missing login checks, missing role checks and access to another user's records.
- Detect and confirm Unsafe Input Handling problems: SQL injection, command injection, path traversal and template injection.
- Run the whole process from VS Code, without the developer typing Docker commands or writing a Dockerfile.
- Never change the developer's real database or files during a scan.
- Show fewer false alarms than a code-only scanner on a benchmark of 10 to 15 Flask applications.

Out of scope for version 1.0: frameworks other than Flask; databases other than SQLite; vulnerability classes other than the two above; scanning of live or remote websites; AI-generated fixes or explanations; and continuous-integration plug-ins.

SecureFlow is a final year project (FYP) at SZABIST University, Karachi Campus. It supports the university's goal of teaching secure software practice, and its results are intended to be compared with Bandit on the same set of applications.

1.5 References

No.	Reference
1	K. Wiegers and J. Beatty, Software Requirements, 3rd ed., Microsoft Press, 2013 (source of the SRS template).
2	SZABIST Karachi, SRS_Template_2025 and Sign Up Use Case example, Faculty of Computing and Engineering Sciences, 2025.
3	S. Munsif and M. Arshad, SecureFlow FYP Proposal (simplified version), SZABIST Karachi, 2026.
4	S. Munsif and M. Arshad, SecureFlow project plan: Semester 1 and Semester 2 Gantt charts, 2026.
5	S. Munsif and M. Arshad, SecureFlow VS Code extension screens (design canvas, 15 boards), 2026.
6	OWASP Foundation, OWASP Top 10: 2021 (A01 Broken Access Control, A03 Injection), https://owasp.org/Top10/.
7	OWASP Foundation, Application Security Verification Standard (ASVS) 4.0.
8	PyCQA, Bandit documentation, https://bandit.readthedocs.io/ (comparison baseline).
9	Pallets Projects, Flask documentation, https://flask.palletsprojects.com/.
10	Docker Inc., Docker Engine documentation, https://docs.docker.com/engine/.
11	Microsoft, Visual Studio Code Extension API, https://code.visualstudio.com/api.
12	SQLite Consortium, SQLite Online Backup API, https://www.sqlite.org/backup.html.
13	IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

Table 1. References

2. Overall Description

2.1 Product Perspective

SecureFlow is a new, self-contained product. It is not part of a larger product family and does not replace an existing system. It sits between two kinds of existing tools: code-only scanners such as Bandit, which are fast but cannot tell whether a problem is real, and full penetration-testing tools, which test live sites but need security expertise to set up. SecureFlow gives the developer a middle path: scan the code, test the suspects safely, and show only proven problems.

The product has three main parts, shown in Figure 1.

- **VS Code extension (TypeScript).** The only part the developer sees. It provides commands, the side panel, the results and detail views, the test setup screen and the editor underlines.
- **Analysis and verification engine (Python).** Reads the code, finds suspects, controls the test box, runs the live tests and builds the findings. The extension starts it as a command-line program and reads its output as JSON that is checked against a schema.
- **Test environment (Docker).** A disposable container that holds a copy of the developer's app and database. It has no internet access while running and is deleted after the scan.

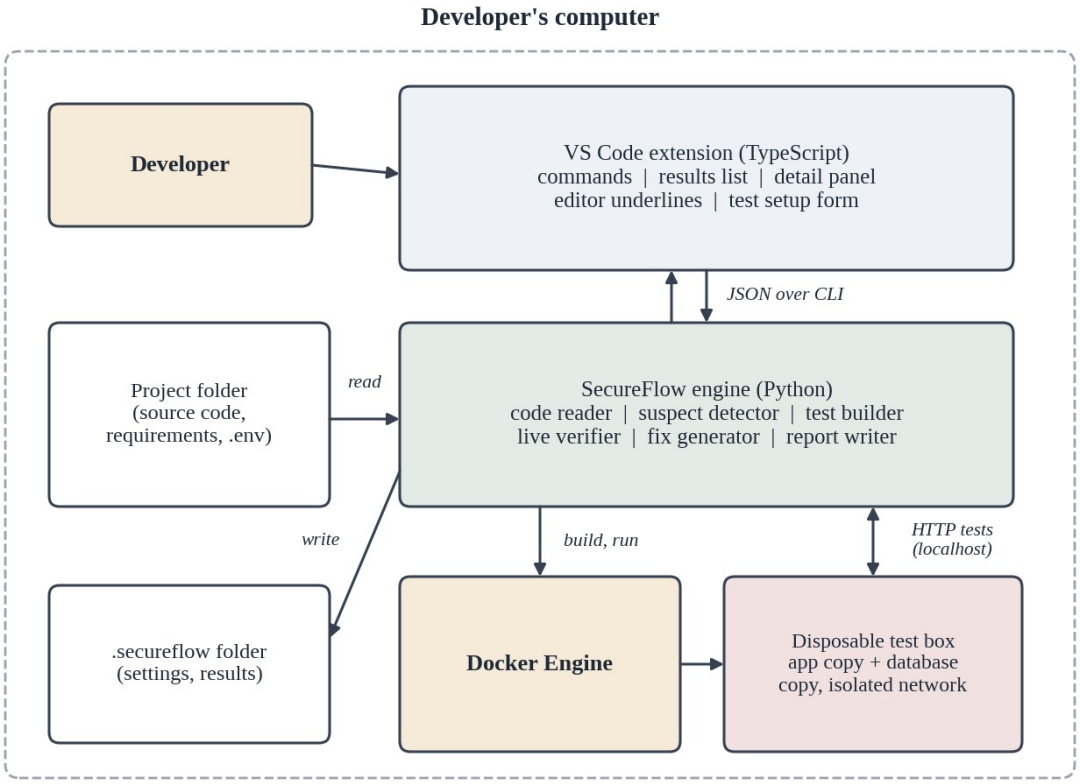


Figure 1. SecureFlow architecture and external interfaces

2.2 Product Functions

The main functions of the product are listed below. Section 4 describes each in detail.

- **4.1 Run Security Scan:** This feature lets the developer start a scan of the open Flask project from VS Code.
- **4.2 Complete First-Run Setup:** The first time a developer scans, SecureFlow prepares the computer once: it checks Docker, downloads the base test image and runs a quick self-check.
- **4.3 Configure Test Setup:** The developer tells SecureFlow how to log in to the app and what data to test with: the test accounts, how the app returns a login (session cookie, token or API key), the database file to copy, the environment file, and folders to skip.
- **4.4 Prepare the Test Environment:** Before any test, the system builds a disposable test box from the developer's project and a copy of the database.
- **4.5 Detect Suspected Vulnerabilities:** The system reads the project's Python files without running them and lists the places that look like broken access control or unsafe input handling.
- **4.6 Verify Suspects with Live Tests:** For each suspect the system sends real test requests to the app in the test box and decides whether the problem is real.
- **4.7 View Scan Results:** After a scan the developer sees a results panel that lists only the problems that were proven, grouped by type and sorted by severity, with counts for suspects, confirmed, discarded and unclear.
- **4.8 View Finding Details and Proof:** For every confirmed problem the developer can open a detail view that explains what was found in plain language and shows the proof: the exact request that was sent and the response that came back.
- **4.9 Apply a Suggested Fix and Re-test:** For common patterns the system can propose a ready-made code fix, show exactly what would change, apply it after the developer agrees, and then run the same test again to prove the attack no longer works.
- **4.10 Export Report:** The developer can save the results of a scan as a report to keep, share with a teammate or attach to a project submission.
- **4.11 Handle Scan Errors and Fallbacks:** When something stops a scan, the system explains the problem in plain words and offers the next step.
- **4.12 View Scan History:** The developer can open a list of earlier scans of the project and compare any two of them.

Figure 10 in Appendix B shows how these functions relate to the developer and to Docker. The work is split over two semesters, as Table 2 shows. Until Docker verification is added in FYP-2, every result is a code-only result and is labelled Unconfirmed.

Feature	FYP-1 (Semester 1)	FYP-2 (Semester 2)
4.1 Run Security Scan	Code scan, progress, cancel, underlines, status bar	Live-testing steps, cleanup, partial results
4.2 Complete First-Run Setup	Not in this phase	Complete
4.3 Configure Test Setup	Not in this phase	Complete
4.4 Prepare the Test Environment	Not in this phase	Complete (Docker, database copy)
4.5 Detect Suspected Vulnerabilities	Complete	Rule tuning from benchmark results
4.6 Verify Suspects with Live Tests	Not in this phase	Complete
4.7 View Scan Results	Suspects list, click-to-code, Unconfirmed labels	Confirmed, discarded and unclear verdicts
4.8 View Finding Details and Proof	Explanation and hover card	Proof (request and response)
4.9 Apply a Suggested Fix and Re-test	Suggestion, preview and apply	Re-test after the fix
4.10 Export Report	HTML and JSON of suspects	Proof and database check added
4.11 Handle Scan Errors and Fallbacks	No Flask app, code-only mode	Docker, build and login errors
4.12 View Scan History	Stored scans, comparison of suspects	Confirmed and fixed states in history
Benchmark and evaluation	Benchmark apps, Bandit baseline, first code-only comparison	Full comparison, user testing, user guide, demo video, final report

Table 2. Feature allocation between FYP-1 and FYP-2

2.3 User Classes and Characteristics

User class	Characteristics	Importance
Student / junior Flask developer (primary)	Builds small Flask apps with a SQLite database. Little or no security training. Comfortable in VS Code. Does not know Docker. Wants a clear answer and a fix.	Most important. All features are designed for this user.
Experienced web developer	Knows Flask and basic security. Wants proof that a flagged problem is real before spending time on it.	Important. Uses the proof and report features most.
Advisor / evaluator	Reviews results, reports and the benchmark comparison with Bandit. Does not run scans on their own code.	Secondary. Uses exported reports.

Table 3. User classes

There is only one role inside the tool: the developer. SecureFlow has no login, no accounts and no administrator. The "test accounts" mentioned in this document belong to the developer's own app, not to SecureFlow.

2.4 Operating Environment

- Computer: a laptop or desktop with at least 8 GB of memory, 4 GB of free disk space and an internet connection for the one-time setup and for building the test image.
- Operating system: Windows 10 or 11 (with Docker Desktop), macOS or Linux. Windows is the main test platform.
- Software: Visual Studio Code 1.85 or later; Docker Engine or Docker Desktop 24 or later; Python 3.10 or later for the engine.
- Target apps: Flask 2.x or 3.x applications written in Python 3.8 or later that use SQLite (directly or through SQLAlchemy).

2.5 Design and Implementation Constraints

- The extension shall be written in TypeScript and the engine in Python. Code analysis shall use Python's own parser (the ast module).
- The extension and engine shall communicate through a command-line interface and JSON that is validated against a schema.
- Live tests shall run only inside a Docker container, never directly on the developer's computer.
- The test box shall have no internet access while running, shall run as a non-root user, shall have CPU and memory limits and shall publish its port on 127.0.0.1 only.
- The product shall work offline after the one-time setup, except when the test image has to be rebuilt with new packages.
- The product shall not need a paid service or an API key. No source code is sent to any external service.
- The project team has two members and two semesters (about 32 weeks), which limits the scope to two vulnerability classes and one framework.
- Passwords entered by the developer shall be kept only in VS Code secret storage.
- SecureFlow shall keep its own data in one local SQLite database file for each project. No database server is used.

2.6 User Documentation

- A short user guide (PDF and README) that covers installation, the first scan, the test setup and how to read the results.
- A troubleshooting page for the common errors listed in Section 4.11.
- In-product help: field hints and plain-language error messages on every screen.
- A demonstration video of a full scan, a fix and a re-test.

2.7 Assumptions and Dependencies

Assumptions

- The developer owns the code being scanned or has permission to test it.
- The developer's app uses SQLite and can run inside a standard Python container.
- The app's login works with an email and password and returns a session cookie, a token or an API key.
- The developer can create two or three test accounts in the app, or use sample data.
- The app does not depend on outside services (payment, email, cloud storage) in a way that stops it starting without them.

Dependencies

- Docker Engine and its image registry for the base Python image.
- The Python package index (PyPI) when the test image is built.
- The VS Code Extension API.
- Open-source Python libraries chosen during design, for example for requests, schema validation and syntax-tree handling.

3. External Interface Requirements

3.1 User Interfaces

The whole interface lives inside VS Code and follows the VS Code look: the same fonts and theme colours, the same side panel and the same editor underlines and hover cards. Every screen uses plain words and avoids security jargon. The main screens are shown below. The data on the screens is sample data.

General rules for every screen:

- Every error message has no more than two sentences and names a next step.
- Every long task shows progress and has a Cancel button.
- Status colours are used together with text or an icon, so the meaning does not depend on colour alone: red for confirmed, amber for being tested or unclear, green for passed or fixed.
- All commands are available from the command palette, and the main ones from the side panel.
- No screen asks for a port, a time limit, an API key or a Docker path.

Command	Screen / Result	Related section
SecureFlow: Scan Project	Progress panel, then results panel	4.1, 4.7
SecureFlow: Open Test Setup	Test setup screen	4.3
SecureFlow: Show Results	Results panel	4.7
SecureFlow: Run Code Scan Only	Results panel labelled Unconfirmed	4.11
SecureFlow: Export Report	Report saved message	4.10

Table 4. Commands in the VS Code command palette

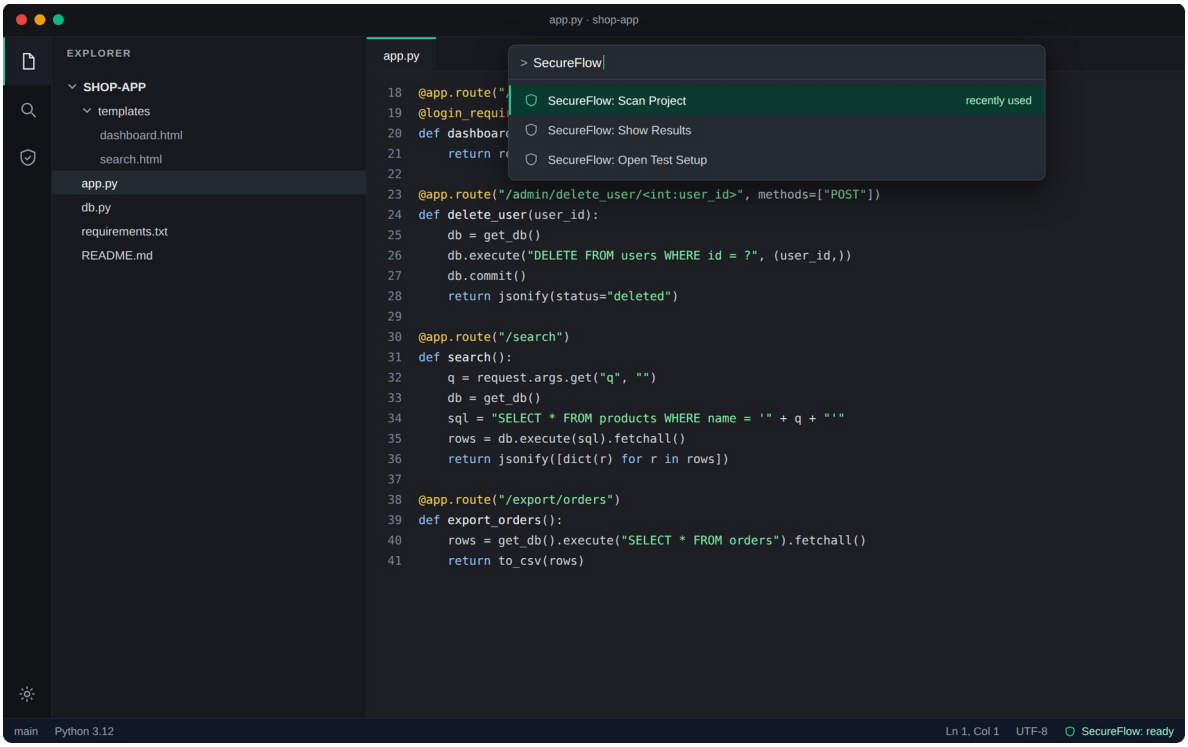


Figure 2. Command palette and editor with the SecureFlow commands

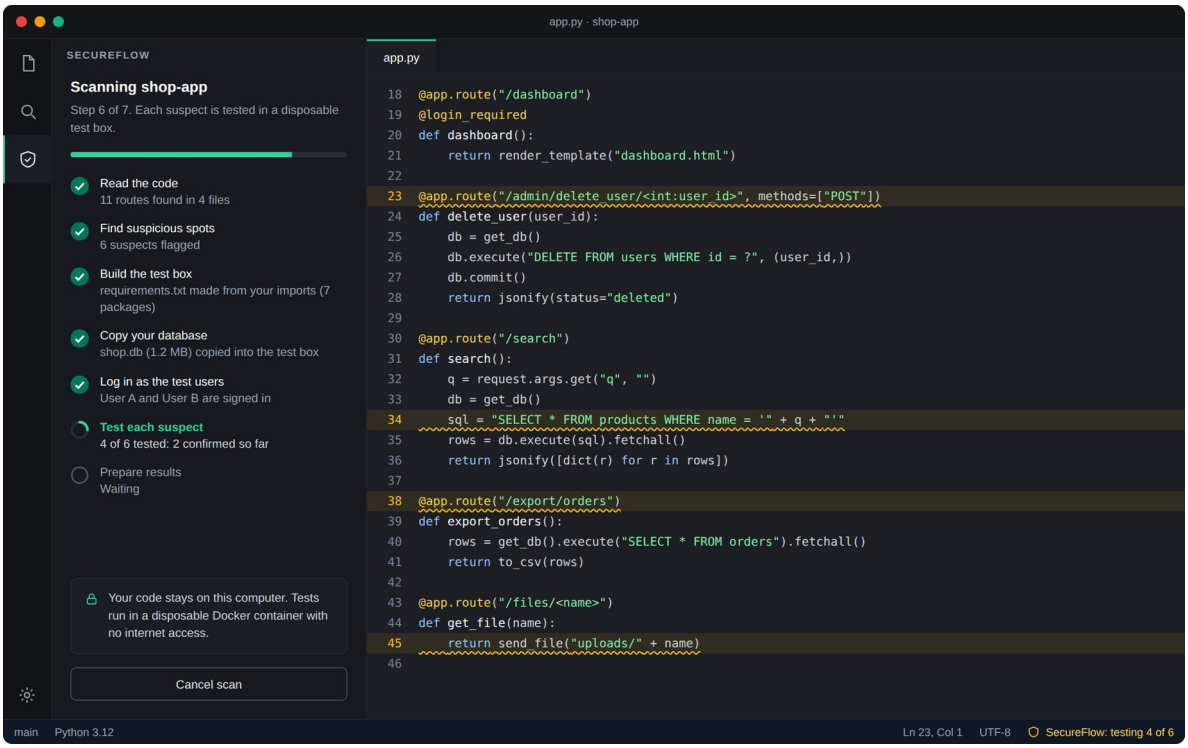


Figure 3. Scan in progress (Section 4.1)

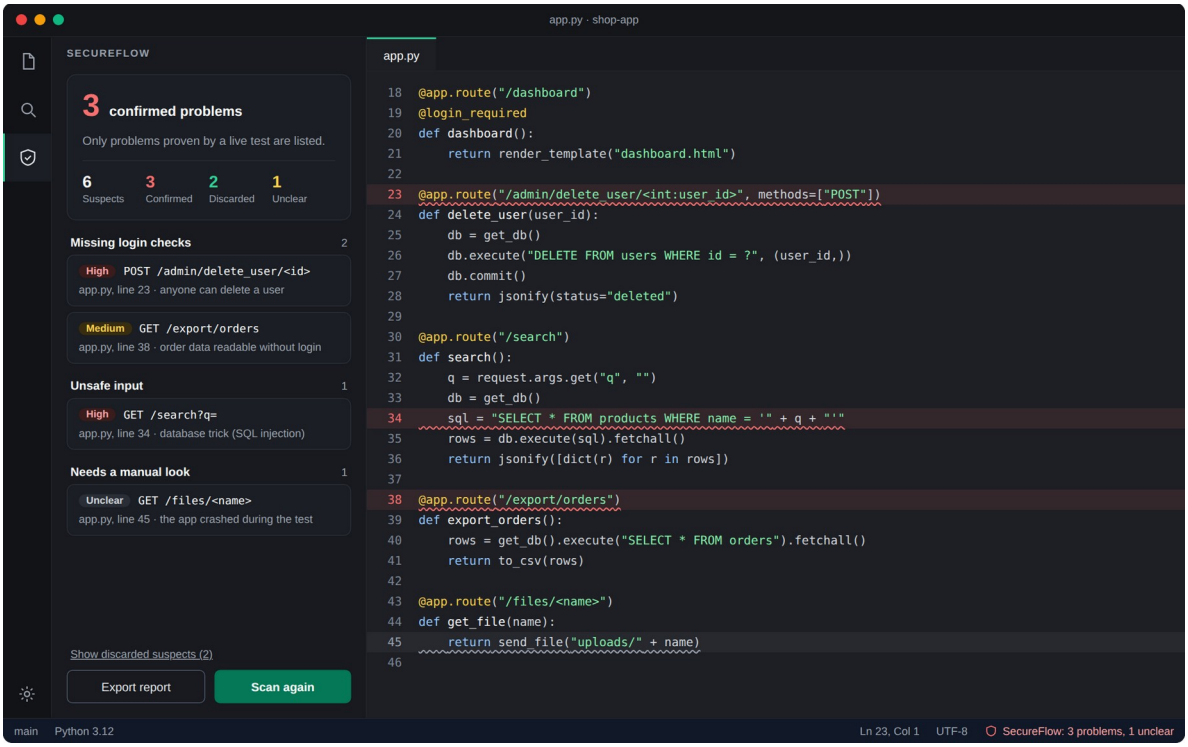


Figure 4. Results panel (Section 4.7)

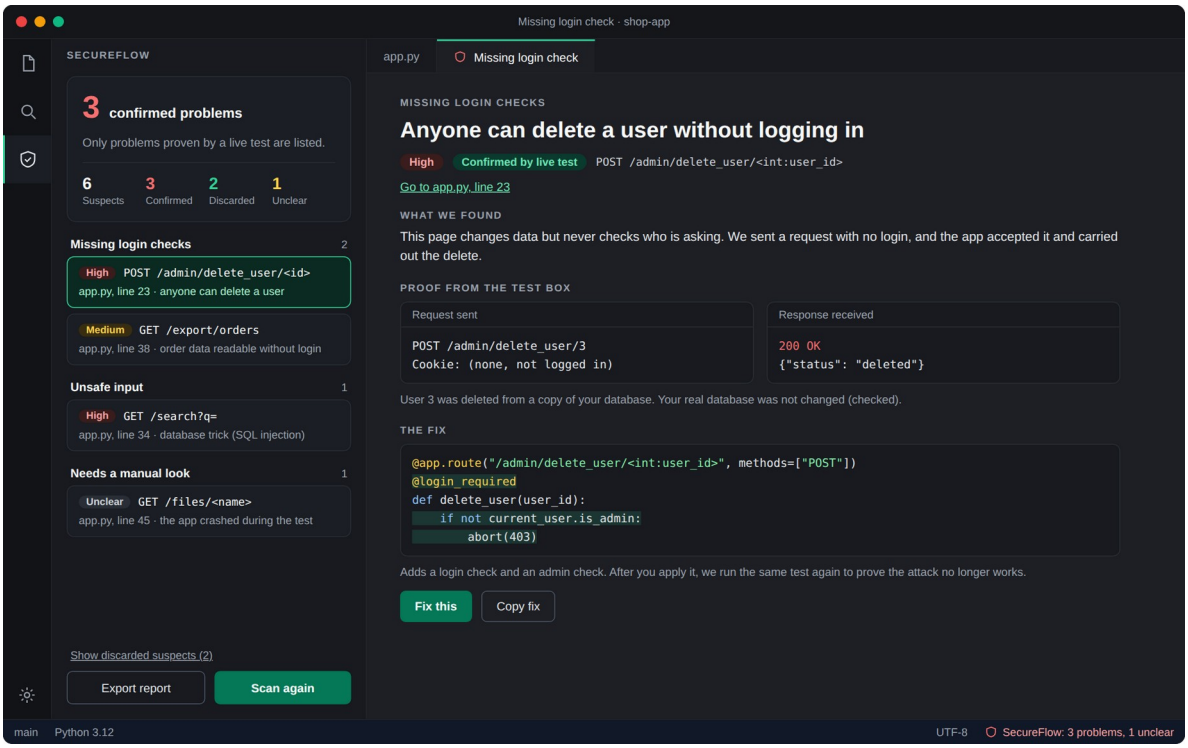


Figure 5. Finding detail with proof (Section 4.8)

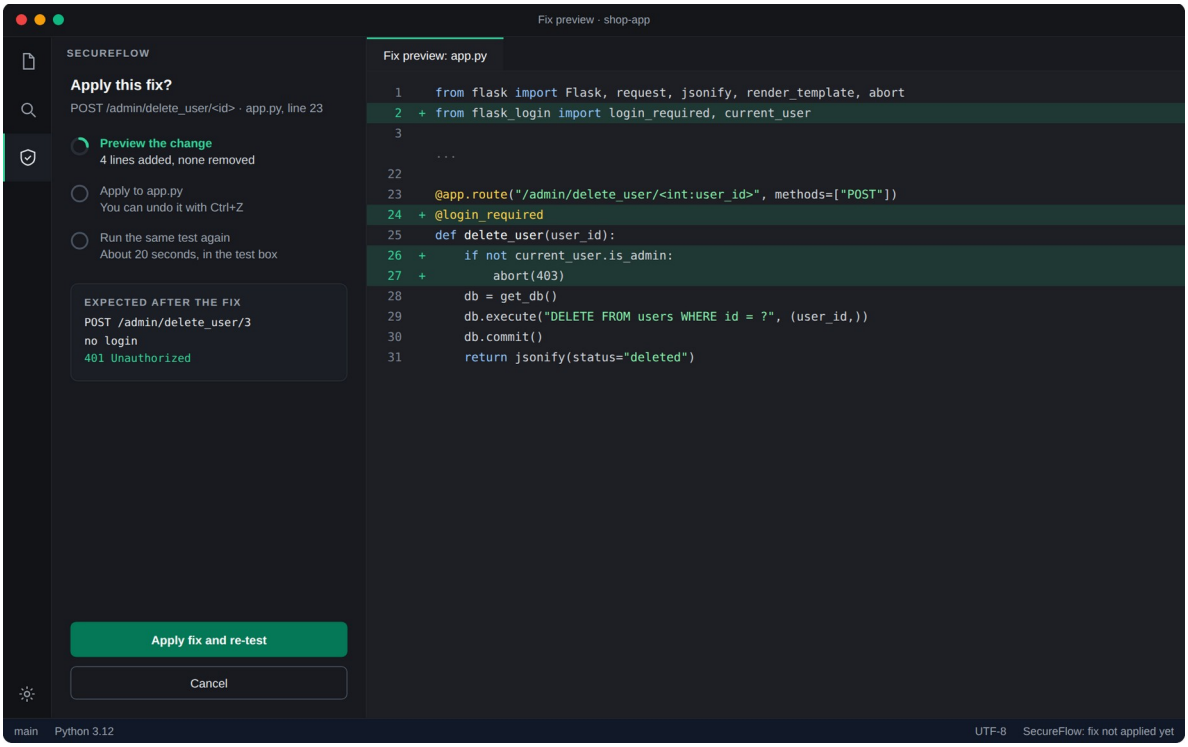


Figure 6. Fix preview (Section 4.9)

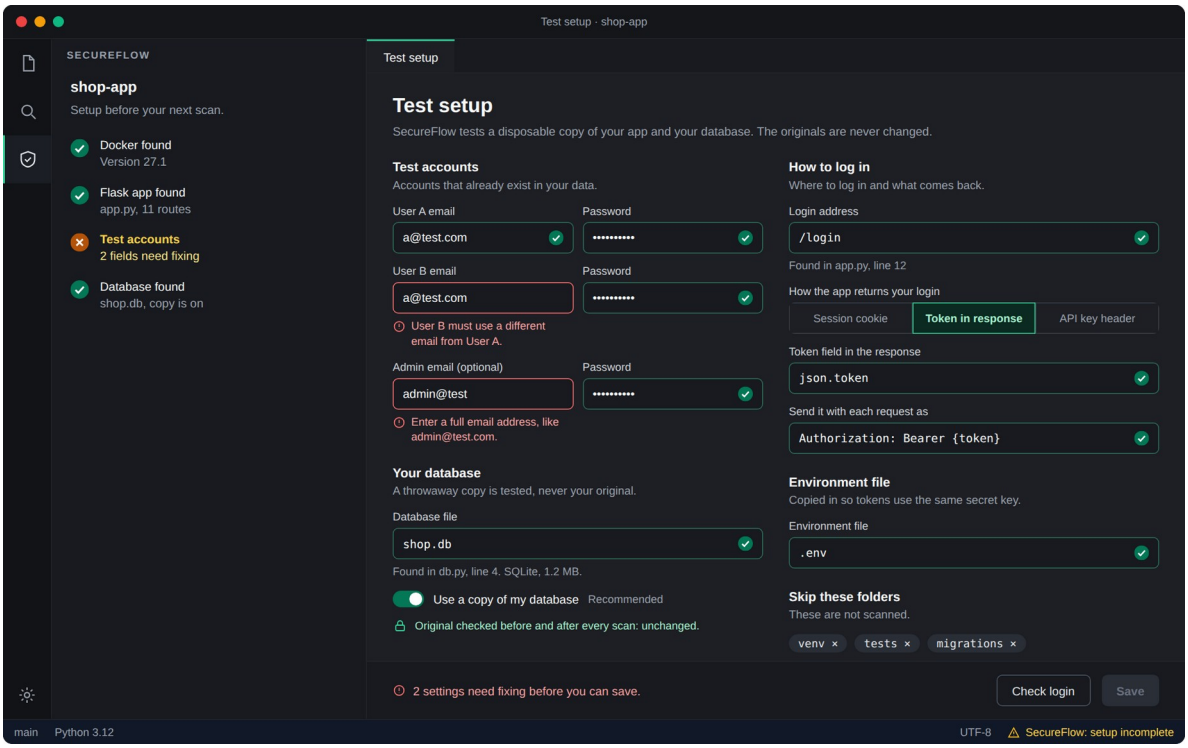


Figure 7. Test setup screen (Section 4.3)

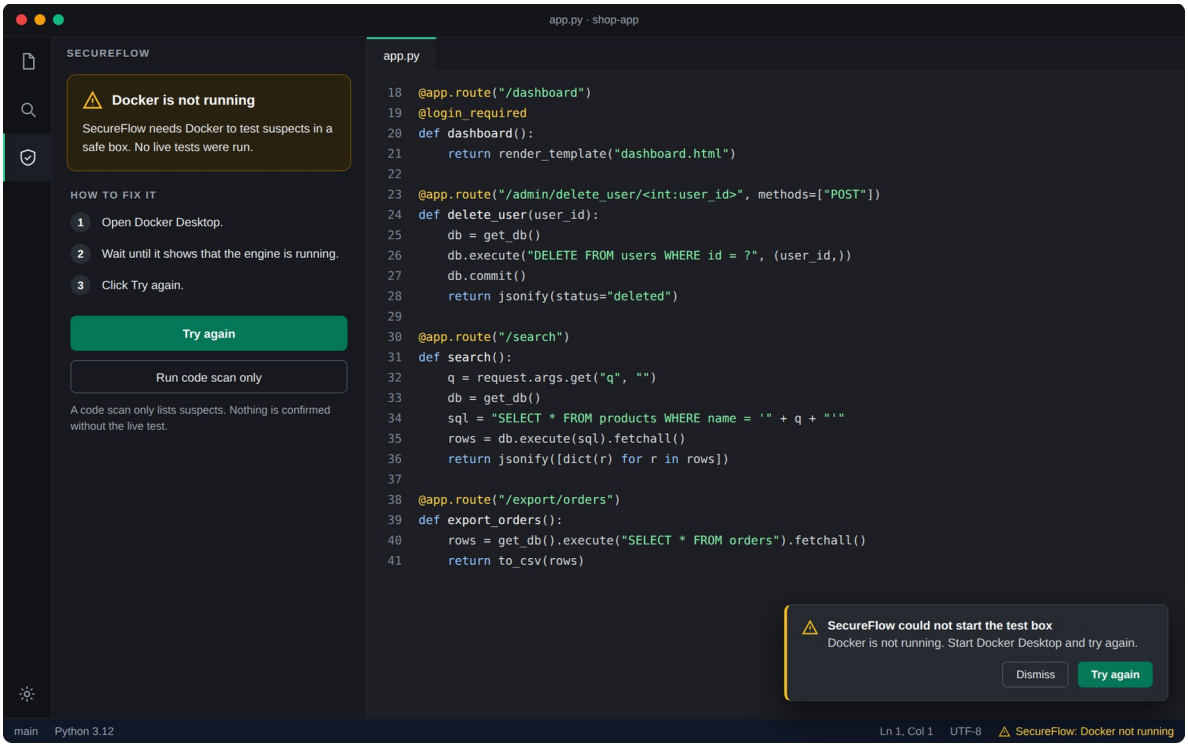


Figure 8. Docker not running (Section 4.11)

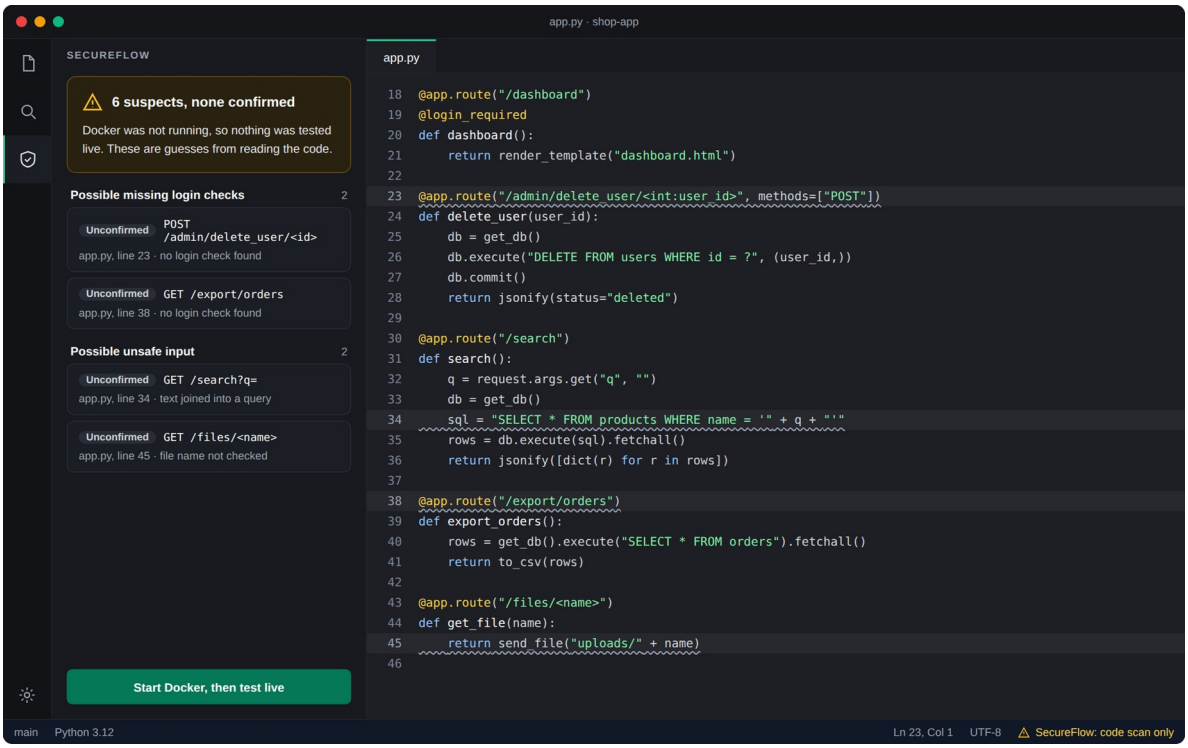


Figure 9. Code-only results labelled Unconfirmed (Section 4.11)

The remaining screens (first-run setup, no problems found, hover card, login failed, build failed and no Flask app found) are in the design canvas listed in the references.

3.2 Hardware Interfaces

SecureFlow has no direct hardware interface. It uses the developer's computer through the operating system for files, network sockets on 127.0.0.1 and the processor and memory shared with Docker. The test box shall be limited so that scanning does not freeze the computer (see requirement ENV-8).

3.3 Software Interfaces

Software	Use	Data exchanged
Visual Studio Code (1.85+)	Host for the extension: commands, panels, diagnostics, hover cards, secret storage.	Commands from the developer; findings and progress to display; stored passwords.
SecureFlow engine (Python 3.10+)	Analysis, test box control, live tests, fix generation.	Command-line arguments in; JSON findings and progress lines out (schema validated).
Docker Engine (24+)	Builds the image, runs and removes the container, sets limits and network.	Build context (project copy, generated Dockerfile, package list, database snapshot); logs and status.
Docker registry	Provides the base Python image once.	Image layers.
PyPI	Provides packages for the app's requirements when the image is built.	Package files.
Flask app under test	Receives HTTP test requests inside the box.	Login requests, test requests and responses on 127.0.0.1.
SQLite (SecureFlow database)	Stores scans, findings, evidence, fixes and logs of each project in one local file, .secureflow/secureflow.db.	Records described in Appendix B.4 and B.5.
SQLite (app database)	Reads the database copy for record IDs and owners; takes a snapshot with the backup function.	Database file copy; hash of the original file.
Operating system file system	Reads project files; writes the .secureflow folder, reports and fixes.	Python files, environment file, report files.

Table 5. Software interfaces

3.4 Communications Interfaces

- HTTP over the local loopback address 127.0.0.1 between the engine and the test box. No test traffic leaves the computer.
- The test box network has no route to the internet while the app runs. The internet is used only to download the base image and the packages during the build.
- Extension to engine: process start with command-line arguments; JSON on standard output and a log file.
- Messages use UTF-8 JSON. Findings contain id, class, subtype, status, route, file, line, severity, reason and evidence.
- No data is sent to Anthropic, OpenAI or any other online service in version 1.0.

4. System Features

This section lists the system features as use cases. For each feature, the sub-section "Description and Priority" gives a short description, "Stimulus/Response Sequences" gives the use case table (user action and system response), and "Functional Requirements" lists the detailed requirements with identifiers, priorities and status. The tables follow the department's Sign Up example. The twelve features are:

- 4.1 Run Security Scan
- 4.2 Complete First-Run Setup
- 4.3 Configure Test Setup
- 4.4 Prepare the Test Environment
- 4.5 Detect Suspected Vulnerabilities
- 4.6 Verify Suspects with Live Tests
- 4.7 View Scan Results
- 4.8 View Finding Details and Proof
- 4.9 Apply a Suggested Fix and Re-test
- 4.10 Export Report
- 4.11 Handle Scan Errors and Fallbacks
- 4.12 View Scan History

4.1 Run Security Scan

4.1.1 Description and Priority

Description: This feature lets the developer start a scan of the open Flask project from VS Code. The scan reads the code, prepares a disposable test box, tests every suspected problem with real requests, and shows only the problems that were proven. It is the main feature of SecureFlow and the one all other features connect to.

Priority: High

Development phase: FYP-1: code scan, progress panel, cancel, editor underlines and status bar. FYP-2: the live-testing steps of the scan, cleanup and partial results.

4.1.2 Stimulus/Response Sequences

Use Case Name: Run Security Scan

Description: The developer starts a scan of the open Flask project and receives the list of problems that live tests have proven.

Actors: Developer (primary). Docker Engine (secondary).

Goal: Find out which suspected security problems in the project are real.

Pre-Conditions: A folder containing a Flask app is open in VS Code. Docker is installed and running. The test setup has been saved (Section 4.3). The first-run setup has been completed (Section 4.2).

Basic Course of Events / Main Flow

1. User Action (Step 1):
The developer opens the command palette and runs "SecureFlow: Scan Project".

System Response:
The system runs the pre-checks (Docker running, Flask app found, test setup saved) and opens the progress panel titled "Scanning <project name>".

2. User Action (Step 2):
The developer waits while the scan runs.

System Response:
The system performs these steps and updates the panel after each one:

- Reads the code and lists routes and suspects (Section 4.5).
- Prepares the test box and copies the database (Section 4.4).
- Logs in as the test users and tests each suspect (Section 4.6).
- The panel shows the current step, a progress bar and counters such as "4 of 6 tested: 2 confirmed so far". Suspects are underlined in amber in the editor while they are being tested.

3. User Action (Step 3):
The developer keeps working or watches the progress.

System Response:
When all suspects are tested, the system:

- Labels each suspect as confirmed, discarded or unclear.
- Removes the test box and the database copy.
- Checks that the original database file is unchanged.
- Saves the results in the project's SecureFlow database.

4. User Action (Step 4):
The developer reads the outcome.

System Response:
The system opens the results panel (Section 4.7), underlines confirmed problems in red in the editor and shows a summary in the status bar, for example "SecureFlow: 3 problems, 1 unclear".

Alternate Flow (Developer Cancels the Scan) **User Action:** The developer clicks "Cancel scan" while the scan is running. **System Response:** The system stops the tests, removes the test box and the database copy, and shows "Scan cancelled". Problems confirmed before the cancel are kept and marked "partial scan". **Alternate Flow (Developer Scans Again)** **User Action:** The developer clicks "Scan again" in the results panel. **System Response:** The system starts a new scan. The previous results stay visible until the new scan finishes, and are then replaced.
Exception Flow (Pre-check Fails or System Error) **System Failure:** Docker is not running, no Flask app is found, the test setup is missing, the build fails, or the login fails. **System Response:** The system stops before testing anything and shows the matching error screen with a next step (Section 4.11). If the engine stops unexpectedly, the system removes the test box and shows "SecureFlow stopped unexpectedly. Your files were not changed. Try again." with an "Open log" button.
Post Conditions: The results panel lists the confirmed problems. Confirmed problems are underlined in the editor. The test box and the database copy are removed. The results are saved in the SecureFlow database.

Table 6. Run Security Scan stimulus/response sequences

4.1.3 Functional Requirements

Req.ID	Description	Priority	Status
SCAN-1	The system shall provide the command "SecureFlow: Scan Project" in the VS Code command palette and a Scan button in the SecureFlow side panel.	High	Pending
SCAN-2	The system shall run pre-checks before every scan: Docker is running, a Flask app is found, and a valid test setup is saved.	High	Pending
SCAN-3	The system shall show a progress panel with the project name, the current step, a progress bar and a count of suspects tested and confirmed so far.	High	Pending
SCAN-4	The system shall let the developer cancel a running scan and shall clean up the test box and database copy within 15 seconds of the cancel.	High	Pending

Req.ID	Description	Priority	Status
SCAN-5	The system shall allow only one scan at a time for a project and shall tell the developer if a scan is already running.	Medium	Pending
SCAN-6	The system shall remove the test box, the built image layers it created and the database copy after every scan, including scans that fail or are cancelled.	High	Pending
SCAN-7	The system shall underline suspects in amber while they are being tested and confirmed problems in red after the scan.	Medium	Pending
SCAN-8	The system shall save each scan and its findings in the project's SecureFlow database (a local SQLite file in the .secureflow folder).	High	Pending
SCAN-9	The system shall show a one-line summary of the last scan in the VS Code status bar.	Low	Pending
SCAN-10	The system shall keep the previous results visible until a new scan finishes successfully.	Low	Pending

Table 7. Run Security Scan functional requirements

4.2 Complete First-Run Setup

4.2.1 Description and Priority

Description: The first time a developer scans, SecureFlow prepares the computer once: it checks Docker, downloads the base test image and runs a quick self-check. Later scans skip this feature. It means the developer never has to type a Docker command.

Priority: Medium

Development phase: FYP-2.

4.2.2 Stimulus/Response Sequences

Use Case Name: Complete First-Run Setup
Description: On the first scan the system prepares everything it needs once, then starts the scan.
Actors: Developer (primary). Docker Engine (secondary).
Goal: Get the computer ready for scanning without typing any commands.
Pre-Conditions: SecureFlow is installed in VS Code. Docker is installed and running. The computer is online.
Basic Course of Events / Main Flow **1. User Action (Step 1):** The developer runs the first scan. **System Response:** The system sees that setup is not complete and shows "Setting up SecureFlow" with three steps: Docker found, Download the test image, Check the test tools. A note says "One-time setup, a few minutes. Later scans start in seconds." **2. User Action (Step 2):** The developer waits and can keep working. **System Response:** The system checks the Docker version, downloads the base image while showing the amount downloaded (for example "48 MB of 120 MB"), and then runs a short self-check of the test tools. **3. User Action (Step 3):** The developer sees the setup finish. **System Response:** The system marks setup as complete, stores a marker on the computer and starts the queued scan automatically.

Alternate Flow (Developer Cancels the Setup) **User Action:** The developer clicks "Cancel" during the download. **System Response:** The system stops the download. The scan is not started and setup stays incomplete, so it begins again on the next scan.
Exception Flow (Download Fails) **System Failure:** The computer is offline or the image source is unavailable. **System Response:** The system shows "Could not download the test image. Check your internet connection and try again." with a "Try again" button. **Exception Flow (Docker Missing)** **System Failure:** Docker is not installed. **System Response:** The system shows "SecureFlow needs Docker to test your app safely" with a link to the Docker download page.
Post Conditions: Setup is marked complete. The queued scan starts.

Table 8. Complete First-Run Setup stimulus/response sequences

4.2.3 Functional Requirements

Req.ID	Description	Priority	Status
FRUN-1	The system shall detect whether first-run setup has been completed on this computer.	High	Pending
FRUN-2	The system shall show the setup steps and the download progress in the side panel.	Medium	Pending
FRUN-3	The system shall download the base test image only if it is not already present.	Medium	Pending
FRUN-4	The system shall start the scan that triggered the setup automatically when setup finishes.	Medium	Pending
FRUN-5	The system shall let the developer cancel the setup and shall resume it on the next scan.	Low	Pending
FRUN-6	The system shall show a clear message with a retry button if the download fails.	High	Pending
FRUN-7	The system shall show a short notice that SecureFlow must only be used on code the developer owns or has permission to test.	Medium	Pending

Table 9. Complete First-Run Setup functional requirements

4.3 Configure Test Setup

4.3.1 Description and Priority

Description: The developer tells SecureFlow how to log in to the app and what data to test with: the test accounts, how the app returns a login (session cookie, token or API key), the database file to copy, the environment file, and folders to skip. Every field is checked as the developer types, and the setup cannot be saved while any field is invalid. This is the feature where input validation is most visible.

Priority: High

Development phase: FYP-2.

4.3.2 Stimulus/Response Sequences

Use Case Name: Configure Test Setup
Description: The developer enters and saves the settings SecureFlow needs to log in to the app and test it.
Actors: Developer.
Goal: Save a valid test setup so that scans can log in and test the app.
Pre-Conditions: A Flask project is open in VS Code.
Basic Course of Events / Main Flow **1. User Action (Step 1):** The developer runs "SecureFlow: Open Test Setup". **System Response:** The system opens the Test setup screen. It fills in the values it found (login address, database file, environment file) and shows a checklist: Docker found, Flask app found, Test accounts, Database found. **2. User Action (Step 2):** The developer enters the email and password for User A and User B, and optionally for an admin. **System Response:** The system checks each field as it is typed and shows a green tick when it is valid or a red message when it is not: – Emails must be in a valid email format. – User A and User B must use different emails. – Passwords must be at least 8 characters. **3. User Action (Step 3):** The developer chooses how the app returns a login: session cookie, token in response, or API key header. **System Response:** The system shows only the fields needed for the chosen type, for example the token field (such as

json.token) and the header format (such as "Authorization: Bearer {token}"). It checks the login address against the routes found in the code.

4. User Action (Step 4):

The developer confirms the database file, keeps "Use a copy of my database" switched on, and sets the environment file and folders to skip.

System Response:

The system checks that the file exists and is an SQLite database, shows its size, and shows the note "Original checked before and after every scan."

5. User Action (Step 5):

The developer clicks "Check login" (optional).

System Response:

The system starts the test box briefly, tries to log in as each account and reports "User A signed in" or the exact reason it failed.

6. User Action (Step 6):

The developer clicks "Save".

System Response:

The system enables Save only when no field has an error. It saves the setup, stores passwords in VS Code secret storage and shows "Test setup saved".

Alternate Flow (Invalid Inputs) **User Action:** The developer enters invalid data, for example an incomplete email, the same email for User A and User B, a short password, a login address that does not exist, or a database file that is not found. **System Response:** The system highlights the fields and shows messages such as: – "Enter a full email address, like admin@test.com." – "User B must use a different email from User A." – "Password must be at least 8 characters." – "Choose an existing SQLite database file." – A banner at the bottom reads "2 settings need fixing before you can save." and the Save button stays disabled. **Alternate Flow (No Database Yet)** **User Action:** The project has no database file, for example a brand-new app. **System Response:** The developer clicks "No database yet? Use sample data" and then "Generate starter file". The system reads the table definitions from the code and writes a sample data file with two users, products and orders for the developer to review.
Exception Flow (Settings Cannot Be Saved) **System Failure:** The project folder is read-only or the disk is full. **System Response:** The system shows "Could not save the test setup. Check that the project folder can be written to." and keeps the entered values on screen.
Post Conditions: The test setup is saved in the project's SecureFlow database. Passwords are stored only in VS Code secret storage.

Table 10. Configure Test Setup stimulus/response sequences

4.3.3 Functional Requirements

Req.ID	Description	Priority	Status
SETUP-1	The system shall provide the command "SecureFlow: Open Test Setup" and open the Test setup screen from the side panel.	High	Pending
SETUP-2	The system shall pre-fill the login address, database file and environment file when it can find them in the code.	Medium	Pending
SETUP-3	The system shall validate emails against a standard email format and shall reject values without a domain, such as "admin@test".	High	Pending

Req.ID	Description	Priority	Status
SETUP-4	The system shall reject a setup where User A and User B have the same email.	High	Pending
SETUP-5	The system shall require passwords of at least 8 characters.	High	Pending
SETUP-6	The system shall offer three login types (session cookie, token in response, API key header) and shall show only the fields that belong to the chosen type.	High	Pending
SETUP-7	The system shall check that the login address starts with "/" and matches a route found in the code, and shall warn if it does not.	Medium	Pending
SETUP-8	The system shall check that the database file exists, is readable and is a valid SQLite database, and shall show its size.	High	Pending
SETUP-9	The system shall switch "Use a copy of my database" on by default and shall show that the original file is checked before and after every scan.	High	Pending
SETUP-10	The system shall warn the developer if the database file is larger than 200 MB.	Low	Pending
SETUP-11	The system shall let the developer choose an environment file and a list of folders to skip.	Medium	Pending
SETUP-12	The system shall provide a "Check login" action that tries to log in as each configured account and reports the result.	Medium	Pending
SETUP-13	The system shall disable Save and show the number of invalid settings while any field has an error.	High	Pending
SETUP-14	The system shall generate a starter sample data file from the tables found in the code when no database exists.	Medium	Pending
SETUP-15	The system shall store passwords only in VS Code secret storage and shall not write them to the settings file, the database, logs or reports.	High	Pending

Table 11. Configure Test Setup functional requirements

4.4 Prepare the Test Environment

4.4.1 Description and Priority

Description: Before any test, the system builds a disposable test box from the developer's project and a copy of the database. It creates the Dockerfile and the package list when they are missing, so the developer needs no Docker or packaging knowledge. The box has no internet access while it runs and is removed after the scan.

Priority: High

Development phase: FYP-2 (the complete Docker setup and the database copy).

4.4.2 Stimulus/Response Sequences

Use Case Name: Prepare the Test Environment
Description: The system builds and starts a disposable copy of the app and its database that can be attacked safely.
Actors: SecureFlow Engine (primary). Docker Engine (secondary). Started by the developer's scan.
Goal: Run a safe copy of the app that can be tested without any risk to the developer's files or data.
Pre-Conditions: The pre-checks have passed and the test setup is saved.
Basic Course of Events / Main Flow **1. Trigger (Step 1):** The scan reaches the build step. **System Response:** The system finds the app's entry file and app object. It uses the project's own Dockerfile if there is one. Otherwise it generates one from a standard template (Python base image, project files, install packages, start command). **2. Trigger (Step 2):** The system needs the list of packages. **System Response:** The system picks the first source available: – requirements.txt if it exists. – pyproject.toml or Pipfile if they exist. – Otherwise a list built from the imports in the code, leaving out Python's built-in modules and the project's own files, and mapping import names to package names. Flask is always included. **3. Trigger (Step 3):** The system prepares the data. **System Response:** The system records the SHA-256 hash of the original database file, takes a consistent snapshot of it with SQLite's backup function, and copies the snapshot, the project files and the environment file into the build. Nothing from the developer's computer is mounted into the box.

4. Trigger (Step 4): The system starts the box. **System Response:** The system builds the image and starts the container on an isolated network with no internet access, with CPU and memory limits, as a non-root user, on a random free port bound to 127.0.0.1. It waits until the app answers an HTTP request (default limit 60 seconds) and reports "App is running and ready".
Alternate Flow (Package List Generated) **Condition:** The project has no requirements.txt or similar file. **System Response:** The system shows "No requirements.txt found. We made one from your imports (7 packages)." The generated file exists only inside the test box. A "Save it to my project" link lets the developer keep it.
Exception Flow (Build or Start Fails) **System Failure:** A package cannot be installed, or the app does not start within the time limit. **System Response:** The system stops, removes the partial image and container, and shows the Build failed screen with the last lines of the build output and the likely cause (Section 4.11).
Post Conditions: A test box is running with the app ready. The database copy is inside it. The hash of the original database file is recorded.

Table 12. Prepare the Test Environment stimulus/response sequences

4.4.3 Functional Requirements

Req.ID	Description	Priority	Status
ENV-1	The system shall detect the app's entry file and app object from the code.	High	Pending
ENV-2	The system shall use the project's Dockerfile if present and otherwise generate one from a standard template.	High	Pending
ENV-3	The system shall get the package list from requirements.txt, then pyproject.toml or Pipfile, then from the imports in the code, in that order.	High	Pending
ENV-4	When building the list from imports, the system shall exclude built-in modules and the project's own modules, map import names to package names using a lookup table, and always include Flask.	High	Pending
ENV-5	The system shall keep a generated package list inside the test box only and shall offer to save it to the project.	Medium	Pending

Req.ID	Description	Priority	Status
ENV-6	The system shall take a consistent snapshot of the SQLite database using the SQLite backup function and shall copy, not mount, it into the test box.	High	Pending
ENV-7	The system shall record the SHA-256 hash of the original database file before the scan and compare it after the scan.	High	Pending
ENV-8	The system shall run the test box with no internet access, as a non-root user, with CPU and memory limits, and without access to the Docker socket.	High	Pending
ENV-9	The system shall publish the app's port only on 127.0.0.1 using a free port chosen at run time.	High	Pending
ENV-10	The system shall wait until the app responds to HTTP requests, with a default limit of 60 seconds, before testing.	High	Pending
ENV-11	The system shall copy the environment file into the test box so that tokens and sessions use the same secret key as the app.	Medium	Pending
ENV-12	The system shall show the last lines of the build output and the likely cause when the build or start fails.	High	Pending
ENV-13	The system shall remove containers and images left over from an earlier scan that stopped unexpectedly, at the start of the next scan.	Medium	Pending

Table 13. Prepare the Test Environment functional requirements

4.5 Detect Suspected Vulnerabilities

4.5.1 Description and Priority

Description: The system reads the project's Python files without running them and lists the places that look like broken access control or unsafe input handling. Each place becomes a suspect with a file, a line, a route and a reason. Suspects are only guesses at this stage. The next feature decides which ones are real.

Priority: High

Development phase: FYP-1.

4.5.2 Stimulus/Response Sequences

Use Case Name: Detect Suspected Vulnerabilities
Description: The system analyses the code and produces a list of suspects for the two vulnerability classes.
Actors: SecureFlow Engine.
Goal: Produce a list of suspects, each with file, line, route, class and reason.
Pre-Conditions: A Flask project folder is available and at least one Python file can be read.
Basic Course of Events / Main Flow **1. Trigger (Step 1):** A scan or a code-only scan starts. **System Response:** The system lists the Python files (leaving out skipped folders) and parses each one into a syntax tree. Files with a syntax error are skipped and listed with the line of the error. **2. Trigger (Step 2):** Parsing is complete. **System Response:** The system finds the routes (from decorators, blueprints and add_url_rule calls), their HTTP methods and parameters, and the protections each route has (login decorator, role check, before_request hook). **3. Trigger (Step 3):** The routes are known. **System Response:** The system applies the access control rules and flags: – A route that changes or returns user data with no login check. – An admin-style route with no role check. – A route that takes a record ID with no check that the record belongs to the user. **4. Trigger (Step 4):** The access control rules are done.

System Response: The system applies the unsafe input rules using taint tracking. It follows data from request sources (query string, form, JSON body, files, headers, URL parameters) to risky places (listed below) when nothing checks or cleans the data on the way: – SQL built by joining strings. – Shell commands. – File paths. – Templates built from strings. **5. Trigger (Step 5):** All rules have run. **System Response:** The system creates one suspect for each match with an ID, class, subtype, route, file, line, severity and reason, checks the list against the findings schema and passes it to the next step.
Alternate Flow (No Suspects Found) **Condition:** None of the rules match. **System Response:** The system reports "No suspects found" and ends the scan with the "No problems found" screen. The test box is not started.
Exception Flow (File Cannot Be Read) **System Failure:** A file has a syntax error, an unreadable encoding or no read permission. **System Response:** The system skips the file, lists it with the reason, and continues. If no Python file can be read, it shows "No Flask app found" (Section 4.11).
Post Conditions: A validated list of suspects exists.

Table 14. Detect Suspected Vulnerabilities stimulus/response sequences

4.5.3 Functional Requirements

Req.ID	Description	Priority	Status
CODE-1	The system shall analyse Python files by parsing them into syntax trees and shall not execute the developer's code during this step.	High	Pending
CODE-2	The system shall find Flask routes defined with decorators, blueprints and add_url_rule, including their methods and URL parameters.	High	Pending
CODE-3	The system shall identify route protections: login decorators, role checks and before_request hooks.	High	Pending

Req.ID	Description	Priority	Status
CODE-4	The system shall flag routes that change or return data without a login check (Broken Access Control).	High	Pending
CODE-5	The system shall flag admin-style routes without a role check and routes that take a record ID without an ownership check.	High	Pending
CODE-6	The system shall track data from request sources to risky places and flag SQL built by string joining, shell commands built from input, unchecked file paths and templates built from strings (Unsafe Input Handling).	High	Pending
CODE-7	The system shall give every suspect an ID, class, subtype, route, file, line, severity and reason.	High	Pending
CODE-8	The system shall validate the suspect list against the findings schema before passing it on, and shall reject malformed data.	High	Pending
CODE-9	The system shall skip files with syntax errors and list them with the reason.	Medium	Pending
CODE-10	The system shall skip the folders listed in the test setup and common non-source folders such as venv and node_modules.	Medium	Pending
CODE-11	The system shall limit false alarms by not flagging routes that are protected by a global before_request login check.	Medium	Pending

Table 15. Detect Suspected Vulnerabilities functional requirements

4.6 Verify Suspects with Live Tests

4.6.1 Description and Priority

Description: For each suspect the system sends real test requests to the app in the test box and decides whether the problem is real. A suspect becomes confirmed only when an attack actually works. If the app behaves safely the suspect is discarded, and if the result cannot be decided it is marked unclear. This feature is what separates SecureFlow from tools that only read code.

Priority: High

Development phase: FYP-2.

4.6.2 Stimulus/Response Sequences

Use Case Name: Verify Suspects with Live Tests
Description: The system tests each suspect against the running test box and records a verdict with proof.
Actors: SecureFlow Engine (primary). Docker Engine (secondary).
Goal: Turn every suspect into a confirmed, discarded or unclear finding with evidence.
Pre-Conditions: The test box is running and the app is ready. The test accounts are configured.
Basic Course of Events / Main Flow **1. Trigger (Step 1):** The test box is ready. **System Response:** The system logs in as User A, User B and, if configured, the admin, using the login recipe from the test setup. It keeps the credentials in memory only. It reads the database copy to find real record IDs and who owns them. **2. Trigger (Step 2):** The logins succeeded. **System Response:** For each suspect the system picks the test for its class: – Missing login: send the request with no login and compare with a logged-in request. – Missing role check: send the request as a normal user to an admin route. – Another user's data: User A asks for a record that belongs to User B. – SQL injection: send a normal value and a test payload, then compare the rows returned or look for a database error. – Command injection: send a payload that creates a harmless marker file or causes a short delay. – Path traversal: request a path with "../" that points at a marker file placed outside the allowed folder. – Template injection: send an expression such as {{7*7}} and look for 49 in the response.

3. Trigger (Step 3): A response is received. **System Response:** The system decides: confirmed if the attack worked, discarded if the app refused it or behaved safely, unclear if the app crashed, timed out or gave an answer that cannot be judged. For tests that change data it also checks the database copy. **4. Trigger (Step 4):** A verdict is reached. **System Response:** The system stores the evidence (request, response and what changed), cuts long values and masks emails and tokens, updates the progress counter and changes the underline colour in the editor.
Alternate Flow (Test Times Out) **Condition:** The app does not answer within the time limit (default 10 seconds). **System Response:** The system marks the suspect unclear with the note "No answer in 10 seconds" and continues with the next suspect.
Exception Flow (App Crashes During a Test) **System Failure:** The app stops answering because of a test. **System Response:** The system restarts the test box once and continues. If the same route crashes the app again, the system marks that route unclear. If Docker itself stops, the scan stops, partial results are saved and an error is shown.
Post Conditions: Every suspect has a verdict and, where applicable, saved evidence.

Table 16. Verify Suspects with Live Tests stimulus/response sequences

4.6.3 Functional Requirements

Req.ID	Description	Priority	Status
VERIFY-1	The system shall log in as each configured test account using the login recipe and shall keep the credentials in memory only.	High	Pending
VERIFY-2	The system shall discover real record IDs and owners by reading the database copy, so tests use existing data.	High	Pending
VERIFY-3	The system shall test a missing login check by sending the request with no login and comparing the response with a logged-in request.	High	Pending
VERIFY-4	The system shall test a missing role check by sending the request as a normal user to an admin route.	High	Pending

Req.ID	Description	Priority	Status
VERIFY-5	The system shall test access to another user's data by requesting a record owned by User B while logged in as User A.	High	Pending
VERIFY-6	The system shall test SQL injection by comparing the response to a normal value with the response to a test payload.	High	Pending
VERIFY-7	The system shall test command injection, path traversal and template injection using harmless markers that do not damage the test box.	Medium	Pending
VERIFY-8	The system shall mark a suspect confirmed only when the test shows the attack worked, and shall mark it discarded when the app refuses or handles it safely.	High	Pending
VERIFY-9	The system shall mark a suspect unclear when the app crashes, times out or the result cannot be judged, and shall record the reason.	High	Pending
VERIFY-10	The system shall apply a time limit of 10 seconds to each test by default.	Medium	Pending
VERIFY-11	The system shall send test requests only to the test box on 127.0.0.1 and shall refuse to send them to any other address.	High	Pending
VERIFY-12	The system shall store the evidence for each finding, cut long values and mask emails, tokens and passwords.	High	Pending
VERIFY-13	The system shall restart the test box once if the app crashes and shall continue with the remaining suspects.	Medium	Pending
VERIFY-14	The system shall check that data changed in the database copy for tests that are expected to change data.	Medium	Pending

Table 17. Verify Suspects with Live Tests functional requirements

4.7 View Scan Results

4.7.1 Description and Priority

Description: After a scan the developer sees a results panel that lists only the problems that were proven, grouped by type and sorted by severity, with counts for suspects, confirmed, discarded and unclear. Clicking a row takes the developer to the line of code. Discarded suspects can be shown on request so the developer can see why they were thrown out.

Priority: High

Development phase: FYP-1: list of suspects grouped by type, with click-to-code, all labelled Unconfirmed. FYP-2: confirmed, discarded and unclear verdicts and counts.

4.7.2 Stimulus/Response Sequences

Use Case Name: View Scan Results
Description: The developer reads the outcome of a scan in a results panel and in the editor.
Actors: Developer.
Goal: Know which problems are confirmed, and where they are in the code.
Pre-Conditions: A scan has finished, or was cancelled with partial results.
Basic Course of Events / Main Flow **1. User Action (Step 1):** The scan finishes, or the developer runs "SecureFlow: Show Results". **System Response:** The system shows a summary card with the number of confirmed problems and the counts of suspects, confirmed, discarded and unclear. Below it, findings are grouped as: – Missing login checks. – Unsafe input. – Needs a manual look. – Each row shows the severity (High or Medium), the route, the file and line, and a one-line summary. **2. User Action (Step 2):** The developer clicks a row. **System Response:** The system opens the file at that line and highlights the row. The finding details are available from the row (Section 4.8). **3. User Action (Step 3):** The developer clicks "Show discarded suspects". **System Response:** The system lists the discarded suspects with the reason for each, for example "/profile returned 401

when not logged in". **4. User Action (Step 4):** The developer clicks "Scan again". **System Response:** The system starts a new scan (Section 4.1).
Alternate Flow (No Confirmed Problems) **User Action:** The scan confirmed nothing. **System Response:** The system shows "No confirmed problems" with the counts and the discarded suspects, so the developer can see what was checked. **Alternate Flow (Code Scan Only)** **User Action:** The results come from a code-only scan. **System Response:** The system labels every item "Unconfirmed" and shows a banner explaining that nothing was tested live (Section 4.11).
Exception Flow (Results Cannot Be Read) **System Failure:** The stored results are missing or damaged. **System Response:** The system shows "The results could not be read. Run a new scan." with a "Scan project" button.
Post Conditions: The developer knows which problems are confirmed and where they are.

Table 18. View Scan Results stimulus/response sequences

4.7.3 Functional Requirements

Req.ID	Description	Priority	Status
RES-1	The system shall show a results panel with the count of confirmed problems and the counts of suspects, confirmed, discarded and unclear findings.	High	Pending
RES-2	The system shall list only confirmed problems as problems, and shall list unclear items separately under "Needs a manual look".	High	Pending
RES-3	The system shall group findings by type and show the severity, route, file, line and a one-line summary for each.	High	Pending
RES-4	The system shall open the file at the flagged line when the developer clicks a finding.	High	Pending

Req.ID	Description	Priority	Status
RES-5	The system shall underline confirmed problems in the editor and add them to the VS Code Problems list.	Medium	Pending
RES-6	The system shall let the developer show the discarded suspects with the reason for each.	Medium	Pending
RES-7	The system shall show a "No confirmed problems" screen when nothing is confirmed.	Medium	Pending
RES-8	The system shall label results from a code-only scan as "Unconfirmed".	High	Pending
RES-9	The system shall show a clear message and a re-scan button if the stored results cannot be read.	Low	Pending

Table 19. View Scan Results functional requirements

4.8 View Finding Details and Proof

4.8.1 Description and Priority

Description: For every confirmed problem the developer can open a detail view that explains what was found in plain language and shows the proof: the exact request that was sent and the response that came back. A short hover card in the editor gives the same information in brief. The aim is that the developer can trust the result without reading security jargon.

Priority: High

Development phase: FYP-1: plain-language explanation and hover card. FYP-2: proof (request and response).

4.8.2 Stimulus/Response Sequences

Use Case Name: View Finding Details and Proof
Description: The developer opens the explanation and proof for one finding.
Actors: Developer.
Goal: Understand what the problem is and see evidence that it is real.
Pre-Conditions: The results contain at least one finding.
Basic Course of Events / Main Flow **1. User Action (Step 1):** The developer hovers over an underlined line in the editor. **System Response:** The system shows a hover card with the finding title, severity, a one-line explanation, a short proof summary, and the actions "View full proof" and "Fix this". **2. User Action (Step 2):** The developer clicks "View full proof", or clicks a finding in the results panel. **System Response:** The system opens a detail tab showing: – The title, severity and the label "Confirmed by live test". – The route and a "Go to code" link. – "What we found" in plain language. – "Proof from the test box" with the request sent and the response received side by side. – A note that the test ran on a copy of the database and the real one was not changed. – "The fix" with a preview and the buttons "Fix this" and "Copy fix". **3. User Action (Step 3):** The developer clicks "Copy fix" or "Go to code". **System Response:** The system copies the fix text to the clipboard, or moves the cursor to the flagged line.

Alternate Flow (Unclear Finding) **User Action:** The developer opens a finding labelled "Unclear". **System Response:** The system shows what happened (for example "the app crashed during the test") and a suggested manual check. It does not show a fix button.
Exception Flow (Finding Data Missing) **System Failure:** The evidence for a finding is missing from the stored results. **System Response:** The system shows "The proof for this finding is not available. Run a new scan."
Post Conditions: The developer has seen the explanation and the proof for the finding.

Table 20. View Finding Details and Proof stimulus/response sequences

4.8.3 Functional Requirements

Req.ID	Description	Priority	Status
DET-1	The system shall show a hover card on underlined lines with the title, severity, one-line explanation and the actions "View full proof" and "Fix this".	Medium	Pending
DET-2	The system shall provide a detail view with the title, severity, confirmation label, route, plain-language explanation and a link to the code.	High	Pending
DET-3	The system shall show the request sent and the response received for every confirmed finding.	High	Pending
DET-4	The system shall state in the detail view that the test ran on a copy of the database.	Medium	Pending
DET-5	The system shall write explanations from fixed templates filled with the route, parameter and proof, and shall not need an external service.	High	Pending
DET-6	The system shall provide "Copy fix" and "Go to code" actions.	Low	Pending
DET-7	The system shall explain unclear findings and suggest a manual check instead of a fix.	Medium	Pending

Table 21. View Finding Details and Proof functional requirements

4.9 Apply a Suggested Fix and Re-test

4.9.1 Description and Priority

Description: For common patterns the system can propose a ready-made code fix, show exactly what would change, apply it after the developer agrees, and then run the same test again to prove the attack no longer works. Fixes are built from rules, not guessed by an AI, so they are predictable and can be checked. If no rule fits, the system says so.

Priority: High

Development phase: FYP-1: rule-based suggestion, preview and apply. FYP-2: re-test after the fix.

4.9.2 Stimulus/Response Sequences

Use Case Name: Apply a Suggested Fix and Re-test
Description: The developer previews a fix, applies it and sees the live test run again.
Actors: Developer (primary). SecureFlow Engine and Docker Engine (secondary).
Goal: Fix a confirmed problem and see proof that the fix works.
Pre-Conditions: A confirmed finding exists and a fix rule applies to it. The file has not changed since the scan.
Basic Course of Events / Main Flow **1. User Action (Step 1):** The developer clicks "Fix this" in the detail view or the hover card. **System Response:** The system opens a fix preview in a side-by-side view with added lines highlighted (for example imports, a @login_required decorator and an admin check). A panel asks "Apply this fix?" and shows the expected result after the fix, such as "401 Unauthorized". **2. User Action (Step 2):** The developer clicks "Apply fix and re-test". **System Response:** The system edits the file, keeping comments and formatting, and adds any imports that are needed. The change is a single undo step in VS Code. **3. User Action (Step 3):** The developer waits. **System Response:** The system runs the same live test again on the updated code. If the attack now fails, it marks the finding "Fixed and re-tested", removes the red underline and updates the counts.

Alternate Flow (Fix Does Not Work) **User Action:** The re-test shows the problem is still there. **System Response:** The system shows "The problem is still there" with the new proof and offers to undo the change or to read the manual advice. **Alternate Flow (No Fix Available)** **User Action:** No rule fits the shape of the code. **System Response:** The system shows "No automatic fix for this one" with plain-language advice. The "Fix this" button is disabled. **Alternate Flow (Developer Cancels)** **User Action:** The developer clicks "Cancel" in the preview. **System Response:** The system closes the preview and does not change any file.
Exception Flow (File Changed Since the Scan) **System Failure:** The developer edited the file after the scan. **System Response:** The system shows "app.py changed since the scan. Scan again before applying the fix." and does not apply the fix.
Post Conditions: The file contains the fix, or is unchanged if the developer cancelled. A fixed finding is marked "Fixed and re-tested".

Table 22. Apply a Suggested Fix and Re-test stimulus/response sequences

4.9.3 Functional Requirements

Req.ID	Description	Priority	Status
FIX-1	The system shall generate fixes from fixed rules for the supported patterns: missing login decorator, missing role check, missing ownership check, parameterised queries, safe command arguments and safe file path joining.	High	Pending
FIX-2	The system shall show a preview of every change, with added and removed lines, before anything is written.	High	Pending
FIX-3	The system shall change a file only after the developer clicks "Apply fix and re-test".	High	Pending

Req.ID	Description	Priority	Status
FIX-4	The system shall keep existing comments and formatting when it edits a file, and shall add needed imports.	Medium	Pending
FIX-5	The system shall make each applied fix one undo step.	Medium	Pending
FIX-6	The system shall run the same live test again after a fix is applied and shall report whether the attack still works.	High	Pending
FIX-7	The system shall mark a finding "Fixed and re-tested" and remove its underline when the re-test passes.	High	Pending
FIX-8	The system shall say "No automatic fix for this one" and disable the fix button when no rule applies.	Medium	Pending
FIX-9	The system shall refuse to apply a fix if the file changed after the scan.	High	Pending
FIX-10	The system shall show new proof and offer an undo if the re-test still finds the problem.	Medium	Pending

Table 23. Apply a Suggested Fix and Re-test functional requirements

4.10 Export Report

4.10.1 Description and Priority

Description: The developer can save the results of a scan as a report to keep, share with a teammate or attach to a project submission. The report contains the same information as the results panel, including the proof, and records that the original database was not changed.

Priority: Medium

Development phase: FYP-1: HTML and JSON report of the suspects. FYP-2: proof, discarded suspects and database check added.

4.10.2 Stimulus/Response Sequences

Use Case Name: Export Report
Description: The developer saves the latest scan results as a report file.
Actors: Developer.
Goal: Keep a readable record of what the scan found.
Pre-Conditions: At least one scan has finished.
Basic Course of Events / Main Flow **1. User Action (Step 1):** The developer clicks "Export report" in the results panel. **System Response:** The system asks for the format (HTML or JSON) and shows the default location, the .secureflow/reports folder. **2. User Action (Step 2):** The developer chooses the format and confirms. **System Response:** The system writes the report with: – A summary and the scan date. – The environment (Docker and Python versions). – Confirmed findings with proof, with long values cut and personal data masked. – Discarded suspects with reasons, and unclear findings. – Fixes applied and re-test results. – The result of the original database check. **3. User Action (Step 3):** The developer sees the confirmation. **System Response:** The system shows "Report saved" with a link that opens the file.

Alternate Flow (No Results Yet) **User Action:** No scan has been run. **System Response:** The "Export report" button is disabled with the hint "Run a scan first."
Exception Flow (File Cannot Be Written) **System Failure:** The chosen folder cannot be written to. **System Response:** The system shows "Could not save the report" and asks the developer to choose another location.
Post Conditions: A report file exists in the chosen location.

Table 24. Export Report stimulus/response sequences

4.10.3 Functional Requirements

Req.ID	Description	Priority	Status
RPT-1	The system shall provide an "Export report" action in the results panel.	Medium	Pending
RPT-2	The system shall export reports in HTML and JSON formats.	Medium	Pending
RPT-3	The system shall include the summary, environment details, confirmed findings with proof, discarded suspects with reasons, unclear findings and fixes applied.	Medium	Pending
RPT-4	The system shall include the result of the original database check in every report.	Medium	Pending
RPT-5	The system shall cut long values and mask emails, tokens and passwords in reports.	High	Pending
RPT-6	The system shall save reports in the .secureflow/reports folder by default and let the developer choose another location.	Low	Pending
RPT-7	The system shall disable the export action until a scan has finished.	Low	Pending

Table 25. Export Report functional requirements

4.11 Handle Scan Errors and Fallbacks

4.11.1 Description and Priority

Description: When something stops a scan, the system explains the problem in plain words and offers the next step. It never reports a problem as confirmed unless a live test proved it. When Docker is not available, the developer can still run a code scan whose results are clearly labelled as unconfirmed.

Priority: High

Development phase: FYP-1: No Flask app found and code-only mode. FYP-2: Docker not running, build failed and login failed screens.

4.11.2 Stimulus/Response Sequences

Use Case Name: Handle Scan Errors and Fallbacks
Description: The system explains what stopped a scan and helps the developer recover.
Actors: Developer.
Goal: Recover from the problem, or still get useful results, without reading technical logs.
Pre-Conditions: A scan was started, or the pre-checks were run, and something prevented it from completing.
Basic Course of Events / Main Flow **1. User Action (Step 1):** The developer runs a scan while Docker is not running. **System Response:** The system stops before testing anything and shows a card "Docker is not running" with three steps (Open Docker Desktop, wait until the engine is running, click Try again). It also shows a message in the editor and a warning in the status bar. **2. User Action (Step 2):** The developer starts Docker and clicks "Try again". **System Response:** The system runs the pre-checks again and, if they pass, starts the scan. **3. User Action (Step 3):** Alternatively, the developer clicks "Run code scan only". **System Response:** The system runs only the code analysis. It shows the suspects labelled "Unconfirmed" with the banner "Docker was not running, so nothing was tested live" and a "Start Docker, then test live" button.

Alternate Flow (No Flask App Found) **User Action:** The open folder has no Flask app. **System Response:** The system shows "No Flask app found" with the files it skipped and why (for example "Python, but it does not use Flask" or "Syntax error on line 12"), and the buttons "Choose another folder" and "See supported apps". **Alternate Flow (Build Failed)** **User Action:** A package cannot be installed or the app does not start. **System Response:** The system shows "Build failed" with the last lines of the build output, the likely cause (for example an import name that is not a package name) and the buttons "Open requirements.txt" and "Try again". **Alternate Flow (Login Failed)** **User Action:** The test box cannot log in as a test account. **System Response:** The system shows "Could not log in as User A" with the request sent, the response received and common causes, and the buttons "Open test setup" and "Try again".
Exception Flow (Unknown Error) **System Failure:** An error occurs that has no specific screen. **System Response:** The system shows a short message that the scan stopped, confirms that the developer's files were not changed and the test box was removed, and offers "Open log".
Post Conditions: The developer knows what went wrong and has a next step. Nothing is reported as confirmed without a live test.

Table 26. Handle Scan Errors and Fallbacks stimulus/response sequences

4.11.3 Functional Requirements

Req.ID	Description	Priority	Status
ERR-1	The system shall show a "Docker is not running" card with steps and the buttons "Try again" and "Run code scan only".	High	Pending
ERR-2	The system shall show a "No Flask app found" card listing skipped files and the reason for each.	Medium	Pending
ERR-3	The system shall show a "Build failed" screen with the last lines of the build output, the likely cause and a retry button.	High	Pending

Req.ID	Description	Priority	Status
ERR-4	The system shall show a "Login failed" screen with the request, the response and common causes.	High	Pending
ERR-5	The system shall offer a code-only scan when Docker is not available and shall label all its results "Unconfirmed".	High	Pending
ERR-6	The system shall never report a problem as confirmed unless a live test proved it.	High	Pending
ERR-7	The system shall use plain language of no more than two sentences in each error message and shall name a next step.	Medium	Pending
ERR-8	The system shall state in every error screen that no files were changed and that the test box was removed, where this applies.	Medium	Pending
ERR-9	The system shall write a log file for each scan and provide an "Open log" button.	Low	Pending
ERR-10	The system shall report engine crashes to the developer and clean up the test box.	High	Pending

Table 27. Handle Scan Errors and Fallbacks functional requirements

4.12 View Scan History

4.12.1 Description and Priority

Description: The developer can open a list of earlier scans of the project and compare any two of them. The comparison shows which suspected problems are new, which are still there and which have gone, so the developer can see progress over time. Every scan is stored in a local database, so nothing is lost when VS Code is closed.

Priority: Medium

Development phase: FYP-1: store scans, list them and compare findings between scans. FYP-2: confirmed, fixed and re-tested states appear in the history.

4.12.2 Stimulus/Response Sequences

Use Case Name: View Scan History
Description: The developer opens the stored scans of the project and compares two of them.
Actors: Developer.
Goal: See how the security results of the project change over time.
Pre-Conditions: At least one scan of the project has been stored in the SecureFlow database.
Basic Course of Events / Main Flow **1. User Action (Step 1):** The developer runs "SecureFlow: Show Scan History". **System Response:** The system opens the history panel and lists the stored scans, newest first. Each row shows the date, the mode (full or code only), the status and the counts of suspects, confirmed and fixed problems. **2. User Action (Step 2):** The developer clicks a scan. **System Response:** The system opens the results of that scan in the results panel (Section 4.7), read from the database. **3. User Action (Step 3):** The developer selects two scans and clicks "Compare". **System Response:** The system matches the findings of the two scans by their finding key (the same route, class and file) and shows three groups: – New: found only in the later scan. – Still there: found in both scans. – Gone: found only in the earlier scan. It is labelled "Fixed and re-tested" if a re-test passed. **4. User Action (Step 4):** The developer clicks "Delete scan" on an old scan (optional).

System Response: The system asks for confirmation, removes the scan and everything stored under it (routes, findings, evidence, fixes and log lines) and updates the list.
Alternate Flow (Only One Scan) **User Action:** The project has only one stored scan. **System Response:** The system shows the scan and the message "Run another scan to compare results." The Compare button is disabled. **Alternate Flow (Different Scan Modes)** **User Action:** The developer compares a code-only scan with a full scan. **System Response:** The system warns that the scans used different modes, because code-only findings are Unconfirmed, and then shows the comparison.
Exception Flow (Database Cannot Be Read) **System Failure:** The database file is missing, locked or damaged. **System Response:** The system shows "Scan history could not be opened." with the buttons "Try again" and "Start a new history". Starting a new history renames the damaged file with the ending .damaged and creates an empty database. The old file is never deleted.
Post Conditions: The developer has seen the stored scans or the comparison. The stored data is unchanged, unless the developer deleted a scan.

Table 28. View Scan History stimulus/response sequences

4.12.3 Functional Requirements

Req.ID	Description	Priority	Status
HIST-1	The system shall provide the command "SecureFlow: Show Scan History" and a history panel.	Medium	Pending
HIST-2	The system shall store every scan (full, code only, cancelled or failed) in the project's SecureFlow database, with its start and end time, mode, status and counts.	High	Pending
HIST-3	The system shall list the stored scans newest first with date, mode, status and counts.	Medium	Pending
HIST-4	The system shall open the results of a stored scan in the results panel.	Medium	Pending

Req.ID	Description	Priority	Status
HIST-5	The system shall give every finding a finding key built from its route, class and file, so the same finding can be matched across scans.	High	Pending
HIST-6	The system shall compare two scans and group the findings as new, still there or gone.	Medium	Pending
HIST-7	The system shall label a gone finding "Fixed and re-tested" when its re-test passed.	Low	Pending
HIST-8	The system shall warn the developer when the two scans being compared used different modes.	Low	Pending
HIST-9	The system shall let the developer delete a scan after confirmation and shall delete all records stored under it.	Medium	Pending
HIST-10	The system shall enforce the relationships between the stored records with foreign keys, so that no finding, evidence or fix is left without its parent.	High	Pending
HIST-11	The system shall never delete or overwrite a damaged database file. It shall rename it and start a new one.	Medium	Pending
HIST-12	The system shall create the database on the first scan and store a schema version number in it so that later versions can upgrade it.	Low	Pending
HIST-13	The system shall keep the database inside the project's .secureflow folder and shall not send its content anywhere.	Medium	Pending

Table 29. View Scan History functional requirements

5. Other Nonfunctional Requirements

The numbers in this section are targets for a typical project of up to 50 Python files and a database of up to 50 MB, on a computer that meets Section 2.4. They are checked during testing (Appendix C, TBD-3).

5.1 Performance Requirements

Req.ID	Description	Priority	Status
PERF-1	After first-run setup, the scan shall begin testing within 60 seconds for a typical project, including image build and app start.	High	Pending
PERF-2	The code analysis step shall finish within 10 seconds for a project of up to 50 Python files.	Medium	Pending
PERF-3	A complete scan of a project with up to 20 suspects shall finish within 5 minutes.	Medium	Pending
PERF-4	The extension shall stay responsive during a scan: the editor shall not freeze for more than 1 second.	High	Pending
PERF-5	The results panel shall open within 2 seconds of the scan finishing.	Low	Pending
PERF-6	Copying the database shall take no more than 30 seconds for a 50 MB database.	Medium	Pending
PERF-7	The test box shall use no more than 1 CPU core and 1 GB of memory by default.	Medium	Pending

Table 30. Performance requirements

5.2 Safety Requirements

SecureFlow runs attacks, so safety means protecting the developer's data, files and computer.

Req.ID	Description	Priority	Status
SAFE-1	The system shall never change, move or delete the developer's original database file. The file hash shall be checked before and after every scan and any difference shall be reported at once.	High	Pending
SAFE-2	The system shall change the developer's source files only when the developer accepts a fix in the preview.	High	Pending
SAFE-3	The system shall run all attack requests inside the test box and shall refuse to send them to any other address.	High	Pending
SAFE-4	The system shall not mount any folder of the developer's computer into the test box.	High	Pending
SAFE-5	The system shall remove the test box and database copy after every scan, even after a crash, and shall remove leftovers at the start of the next scan.	High	Pending
SAFE-6	The system shall use harmless markers in command and file tests and shall never run destructive commands.	High	Pending
SAFE-7	The first-run setup shall show a notice that the tool is for use only on code the developer owns or may test.	Medium	Pending

Table 31. Safety requirements

5.3 Security Requirements

Req.ID	Description	Priority	Status
SEC-1	The system shall store test account passwords only in VS Code secret storage, and shall not write them to settings, the database, logs, reports or the test box image.	High	Pending
SEC-2	The system shall mask emails, tokens, passwords and API keys in evidence, logs and reports.	High	Pending
SEC-3	The test box shall have no internet access while the app runs, shall run as a non-root user and shall not have access to the Docker socket.	High	Pending
SEC-4	The port of the test box shall be published on 127.0.0.1 only, so other computers on the network cannot reach it.	High	Pending
SEC-5	The system shall not send source code, data or results to any external service.	High	Pending
SEC-6	The system shall validate all JSON from the engine against a schema and reject malformed data.	Medium	Pending
SEC-7	The system shall treat code and data in the scanned project as untrusted. The analysis step shall not execute it.	High	Pending
SEC-8	The system shall not need a user account, login or registration of its own.	Low	Pending

Table 32. Security requirements

5.4 Software Quality Attributes

Req.ID	Description	Priority	Status
QUAL-1	Usability: a developer who has never used the tool shall be able to finish the first scan, including setup, in 15 minutes using only the on-screen guidance (checked with at least 5 testers).	High	Pending
QUAL-2	Correctness: on the benchmark, the share of confirmed findings that are real problems shall be at least 90%.	High	Pending
QUAL-3	False alarms: on the same benchmark, SecureFlow shall report fewer false positives than Bandit.	High	Pending
QUAL-4	Reliability: no scan shall leave a container, image or database copy behind after it ends, in 20 consecutive runs including cancelled and failed ones.	High	Pending
QUAL-5	Robustness: the system shall show a clear message and recover for each error listed in Section 4.11 without needing VS Code to be restarted.	Medium	Pending
QUAL-6	Maintainability: each detection rule and each fix rule shall be one small, separate unit with its own tests, so a new rule can be added without changing other rules.	Medium	Pending
QUAL-7	Testability: each requirement in Section 4 shall be checked by at least one test case; the engine shall have automated unit tests.	Medium	Pending
QUAL-8	Portability: the product shall run on Windows, macOS and Linux.	Medium	Pending
QUAL-9	Reproducibility: scanning the same project twice with the same setup shall give the same confirmed findings.	Medium	Pending

Table 33. Software quality attributes

Preferences: ease of use is preferred over speed; safety of the user's data is preferred over coverage; a smaller number of proven findings is preferred over a larger number of unproven ones.

5.5 Business Rules

- BR-1: SecureFlow may be used only on code that the developer owns or has written permission to test.
- BR-2: A problem is called "confirmed" only when a live test proved it. Anything else is "unclear" or "unconfirmed".
- BR-3: A fix is applied only after the developer sees the preview and agrees.
- BR-4: The tool is open to every developer on the machine. There are no roles and no permissions inside the tool.
- BR-5: The project is a university final year project and is released for academic use. The licence is TBD (Appendix C).

6. Other Requirements

Database requirements. SecureFlow keeps its own data (test setups, scans, routes, findings, evidence, fixes, database checks, reports and logs) in one local SQLite file, .secureflow/secureflow.db, inside each project. The structure is given in the ERD in Appendix B.4 and the data dictionary in Appendix B.5. Exported reports are separate HTML and JSON files. Foreign keys are enforced, and the database stores a schema version. Passwords are not stored in it. The scanned app's own database is a separate matter: only SQLite databases of the scanned app are supported in version 1.0.

Legal and ethical requirements. The tool performs attack-style tests, so it shall be used only on code the developer is allowed to test, and shall target only the local test box. Open-source libraries used shall have licences that allow academic use.

Internationalisation. The user interface and messages are in English only. Text in the scanned app may be in any language.

Reuse. Detection rules, fix rules and test rules are separate units so they can be reused or extended later for other vulnerability classes.

Evaluation requirements. The project shall include a benchmark of 10 to 15 Flask applications (including intentionally vulnerable ones). The results shall be compared with Bandit on detection rate and false positives.

Appendix A: Glossary

Term	Meaning
API	Application Programming Interface. A way for programs to talk to each other.
AST	Abstract Syntax Tree. A tree form of source code that a program can read without running the code.
Bandit	An open-source Python tool that scans code for security problems without running it. Used as the comparison baseline.
Broken Access Control	A weakness where a user can do something they should not, such as viewing another user's data or using an admin function.
Command injection	A weakness where user input is used to build an operating system command and an attacker can run their own commands.
Confirmed	A suspect for which a live test showed that the attack worked.
Container	A running, isolated environment started from a Docker image.
Discarded	A suspect for which a live test showed that the app is safe.
Docker	A tool that runs programs in isolated containers.
Dockerfile	A text file with the steps to build a Docker image. SecureFlow generates it automatically when the project has none.
Evidence	The request, response and database change saved as proof for a finding.
False positive	A reported problem that is not a real problem.
Finding	A suspect with its verdict and evidence.
FYP	Final Year Project.
Image	A packaged copy of an app and its environment, from which containers are started.
JSON	JavaScript Object Notation. A text format for structured data.
JWT	JSON Web Token. A signed token that some apps use as a login.
Live test	A real request sent to the running app in the test box to check a suspect.
Path traversal	A weakness where input such as "../" lets an attacker read files outside the allowed folder.
requirements.txt	A file that lists the Python packages an app needs.

Term	Meaning
Route	A web address of the app (for example /profile) handled by a Python function.
SQL injection	A weakness where user input changes the meaning of a database query.
ERD	Entity Relationship Diagram. A diagram of the stored data items and how they are linked.
Finding key	A stable key made from a finding's route, class and file. It is used to match the same finding in different scans.
Scan history	The list of earlier scans of a project, kept in the SecureFlow database.
SQLite	A small database stored in a single file.
SRS	Software Requirements Specification.
Suspect	A place in the code that looks like a security problem and has not yet been tested.
Taint tracking	Following data from its source (for example a form field) to a risky place (for example a database query) through the code.
Template injection	A weakness where user input is used as part of a page template and an attacker can run code or read data.
Test box	SecureFlow's name for the disposable Docker container that holds a copy of the app and database.
Unclear	A suspect whose live test could not be decided, for example because the app crashed.
Unconfirmed	A suspect from a code-only scan that has not been tested live.
VS Code	Visual Studio Code, the code editor in which SecureFlow runs.

Table 34. Glossary

Appendix B: Analysis Models

B.1 Use Case Diagram

The developer is the only human actor. Docker Engine is a secondary actor used by the scan, the first-run setup and the fix re-test.

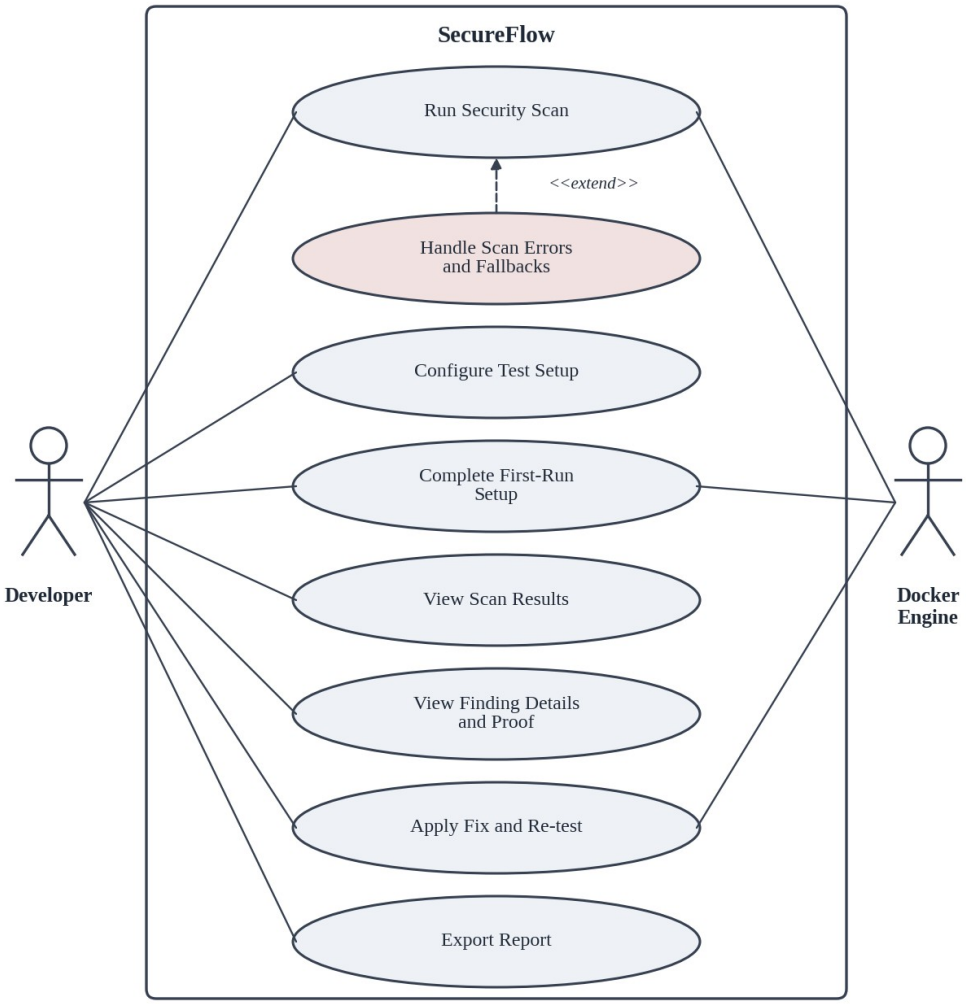


Figure 10. Use case diagram of SecureFlow

B.2 Scan Activity Flow

Figure below shows the order of steps in a complete scan, including the decisions that lead to the error screens.

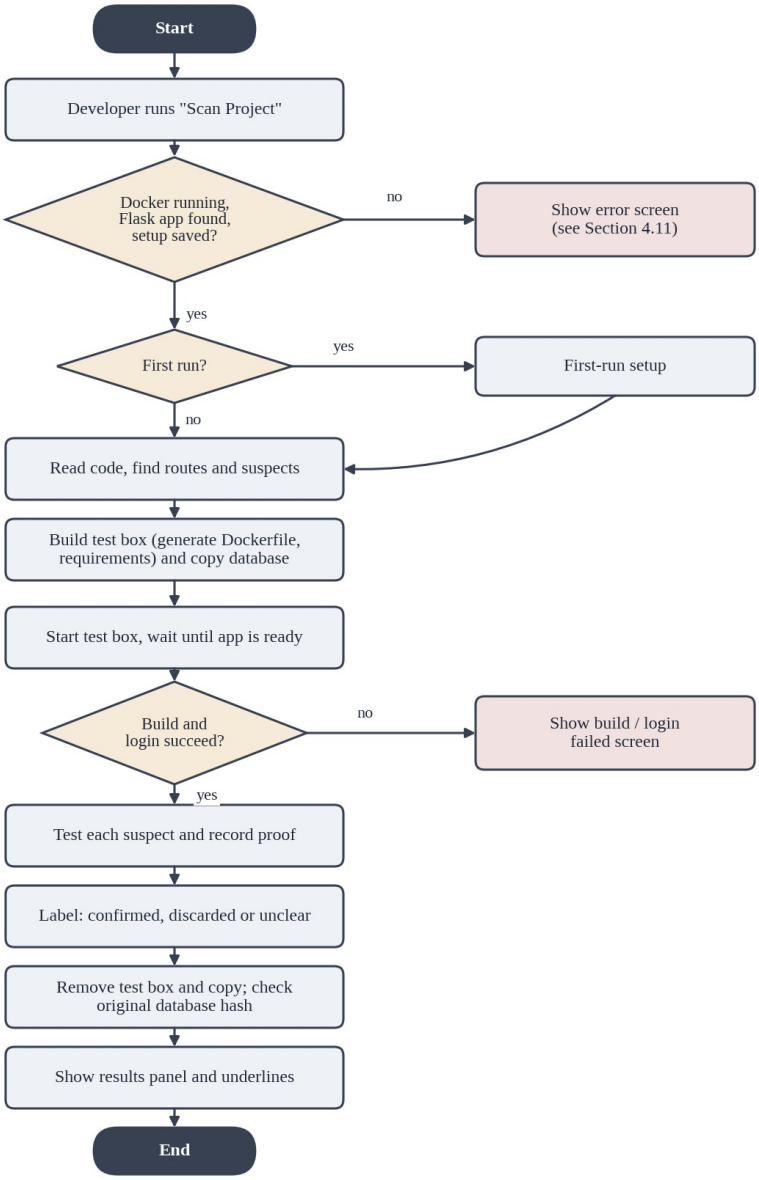


Figure 11. Activity diagram of a scan

B.3 Finding State Diagram

A finding starts as a suspect and moves to one verdict. Only the live test can confirm a suspect.

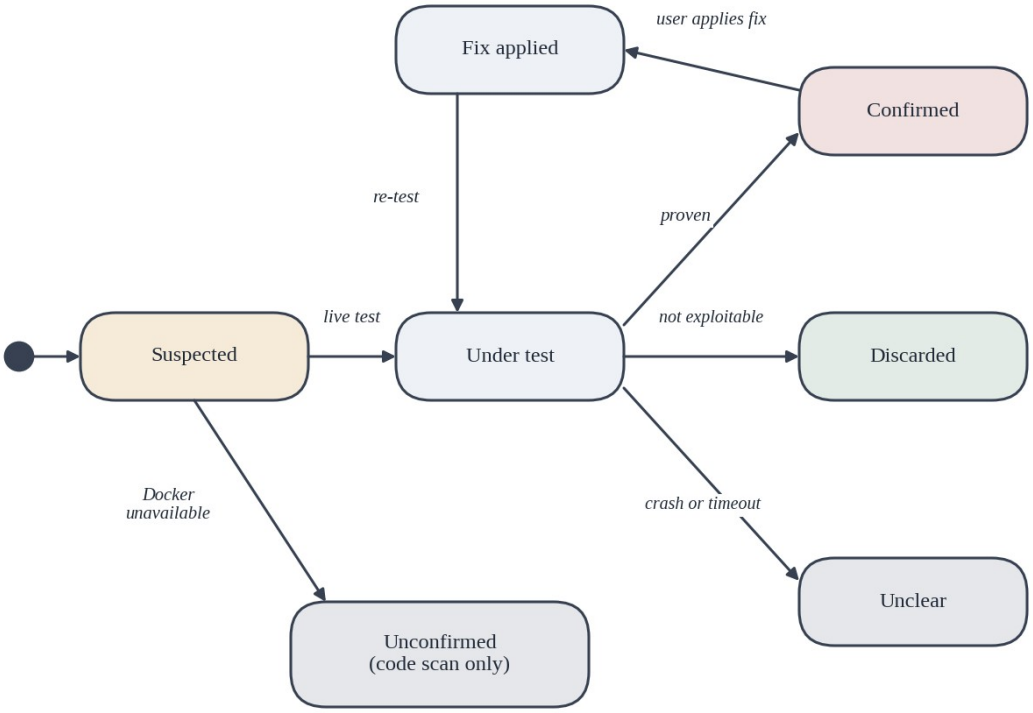


Figure 12. States of a finding

B.4 Entity Relationship Diagram

The SecureFlow database has eleven tables. Passwords are not stored; a test account keeps only the name of its entry in VS Code secret storage.

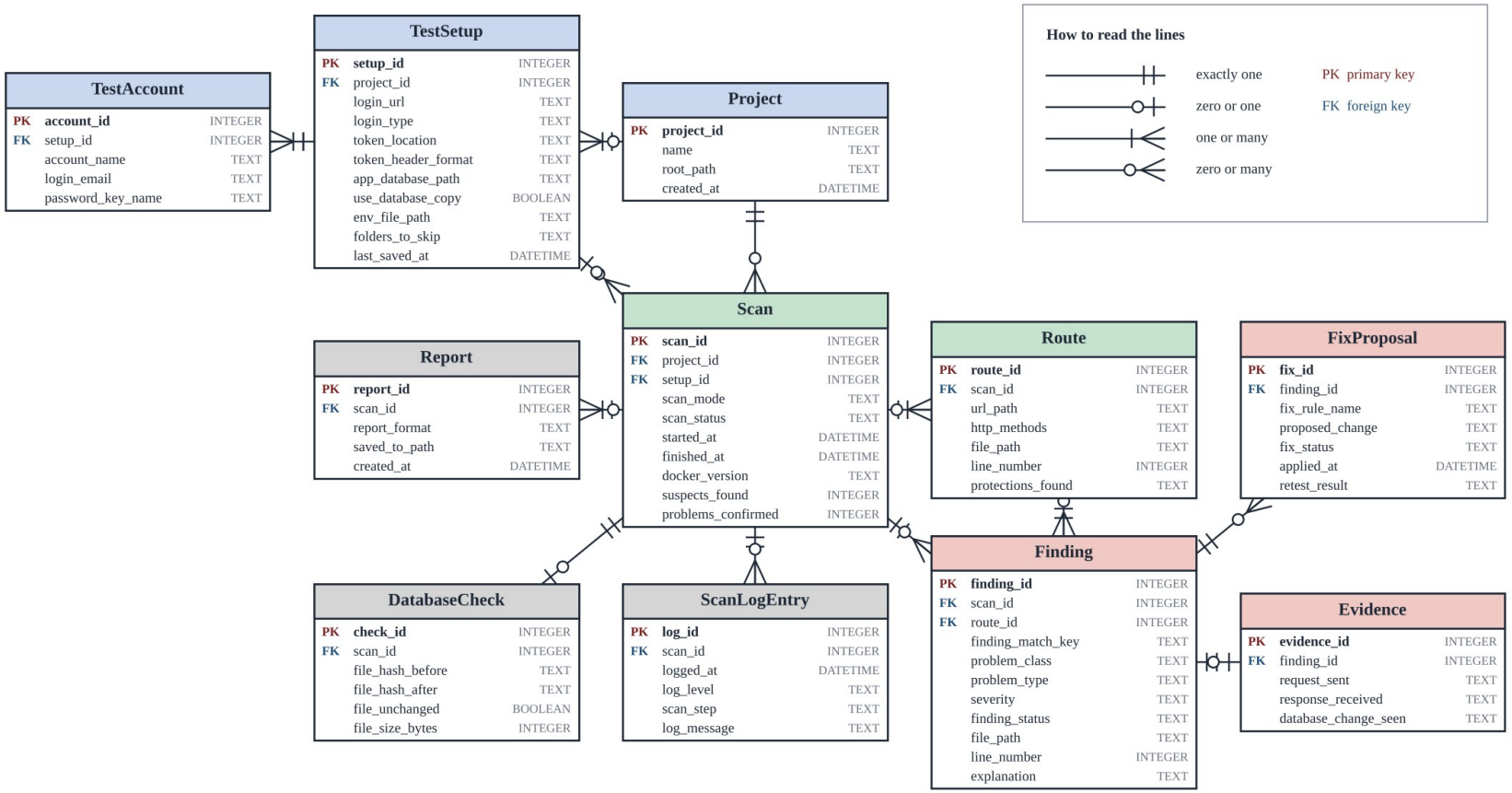


Figure 13. Entity relationship diagram of the SecureFlow database

B.5 Data Dictionary

What the database does: it remembers every scan SecureFlow has run on a project, what was found, the proof, any fixes and the settings that were used. What it does not do: it holds no user accounts, no passwords and none of the scanned app's own data. The scanned app's database is only read through a copy, and the original is never changed.

Every table is listed with all of its fields. Dates are stored as text in ISO format, and true or false values as 0 or 1. Fields marked PK are primary keys and fields marked FK are foreign keys.

Project: One Flask project that has been scanned (the folder open in VS Code).

Field	Type	Key	Description
project_id	INTEGER	PK	Unique number of the project.
name	TEXT		Folder name of the project, for example my-shop.
root_path	TEXT		Where the project folder is on the computer.
created_at	DATETIME		When the project was first scanned.

Table 35. Project table

TestSetup: The saved settings that tell SecureFlow how to log in to the app and which data to use.

Field	Type	Key	Description
setup_id	INTEGER	PK	Unique number of the setup.
project_id	INTEGER	FK	The project this setup belongs to.
login_url	TEXT		Login address of the app, for example /login.
login_type	TEXT		How the app remembers a user after login: cookie, token or api_key.
token_location	TEXT		Where the token is in the login reply, for example json.token. Used only for token logins.
token_header_format	TEXT		How the token is sent back to the app, for example Authorization: Bearer {token}. Used only for token logins.
app_database_path	TEXT		Path of the app's own SQLite database file, so SecureFlow can copy it.

Field	Type	Key	Description
use_database_copy	BOOLEAN		True when tests run on a copy of the app's database. This is the default.
env_file_path	TEXT		Path of the app's environment file (secret keys and settings), copied into the test box.
folders_to_skip	TEXT		Folders left out of the scan, for example venv.
last_saved_at	DATETIME		When the settings were last saved.

Table 36. TestSetup table

TestAccount: A test user of the app: User A, User B or the admin.

Field	Type	Key	Description
account_id	INTEGER	PK	Unique number of the account.
setup_id	INTEGER	FK	The setup this account belongs to.
account_name	TEXT		User A, User B or Admin. A and B are two ordinary users, used to check whether A can reach B's data.
login_email	TEXT		The email this test user logs in with in the app.
password_key_name	TEXT		Name of the entry in VS Code secure storage that holds the password. The password itself is never stored in the database.

Table 37. TestAccount table

Scan: One run of SecureFlow on a project.

Field	Type	Key	Description
scan_id	INTEGER	PK	Unique number of the scan.
project_id	INTEGER	FK	The project that was scanned.
setup_id	INTEGER	FK	The setup used. Empty for code-only scans.
scan_mode	TEXT		full (with Docker live tests) or code_only.
scan_status	TEXT		completed, cancelled or failed.

Field	Type	Key	Description
started_at	DATETIME		When the scan started.
finished_at	DATETIME		When the scan ended.
docker_version	TEXT		Docker version used, shown in reports. Empty for code-only scans.
suspects_found	INTEGER		How many suspicious spots the code analysis found.
problems_confirmed	INTEGER		How many of them the live tests proved to be real.

Table 38. Scan table

Route: A web address of the app found in the code during a scan.

Field	Type	Key	Description
route_id	INTEGER	PK	Unique number of the route.
scan_id	INTEGER	FK	The scan that found the route.
url_path	TEXT		The address, for example /profile/<id>.
http_methods	TEXT		How the address is called: GET reads data, POST sends data.
file_path	TEXT		The code file that defines the route.
line_number	INTEGER		Line of the route in that file.
protections_found	TEXT		Checks found on the route in the code: login required, admin only or owner check.

Table 39. Route table

Finding: A suspect and what the scan decided about it.

Field	Type	Key	Description
finding_id	INTEGER	PK	Unique number of the finding.
scan_id	INTEGER	FK	The scan that produced the finding.
route_id	INTEGER	FK	The route the finding is on.

Field	Type	Key	Description
finding_match_key	TEXT		A fixed label built from route, problem class and file. It lets SecureFlow recognise the same problem in a later scan.
problem_class	TEXT		access_control or unsafe_input.
problem_type	TEXT		The specific problem, for example missing_login or sql_injection.
severity	TEXT		High, Medium or Low.
finding_status	TEXT		suspect, unconfirmed, confirmed, discarded, unclear or fixed.
file_path	TEXT		The code file with the flagged line.
line_number	INTEGER		The flagged line.
explanation	TEXT		One plain sentence on why it was flagged and why it got this status.

Table 40. Finding table

Evidence: The proof saved for a finding.

Field	Type	Key	Description
evidence_id	INTEGER	PK	Unique number of the evidence record.
finding_id	INTEGER	FK	The finding this proves. At most one record per finding.
request_sent	TEXT		The test request SecureFlow sent to the app. Long values are cut and emails, tokens and passwords are hidden.
response_received	TEXT		What the app answered, cut and hidden in the same way.
database_change_seen	TEXT		What changed in the database copy because of the test, for example User 3 deleted. Empty if nothing changed.

Table 41. Evidence table

FixProposal: A suggested code change for a finding.

Field	Type	Key	Description
fix_id	INTEGER	PK	Unique number of the fix.
finding_id	INTEGER	FK	The finding the fix is for.
fix_rule_name	TEXT		Which fix rule produced the change, for example add_login_required.
proposed_change	TEXT		The suggested change shown as added and removed lines.
fix_status	TEXT		proposed, applied, cancelled or undone.
applied_at	DATETIME		When the developer applied the fix. Empty if not applied.
retest_result	TEXT		Result of running the same test again after the fix: passed, failed or not_run.

Table 42. FixProposal table

DatabaseCheck: The before-and-after check of the developer's original database file.

Field	Type	Key	Description
check_id	INTEGER	PK	Unique number of the check.
scan_id	INTEGER	FK	The scan the check belongs to. One check per scan.
file_hash_before	TEXT		A fingerprint (SHA-256 hash) of the original database file before the scan.
file_hash_after	TEXT		The same fingerprint after the scan.
file_unchanged	BOOLEAN		True when both fingerprints are the same, which proves the original file was not changed.
file_size_bytes	INTEGER		Size of the original database file.

Table 43. DatabaseCheck table

Report: An exported report file.

Field	Type	Key	Description
report_id	INTEGER	PK	Unique number of the report.
scan_id	INTEGER	FK	The scan the report is about.
report_format	TEXT		html or json.
saved_to_path	TEXT		Where the report file was saved.
created_at	DATETIME		When the report was exported.

Table 44. Report table

ScanLogEntry: One line of the scan log.

Field	Type	Key	Description
log_id	INTEGER	PK	Unique number of the log line.
scan_id	INTEGER	FK	The scan the line belongs to.
logged_at	DATETIME		Time of the line.
log_level	TEXT		info, warning or error.
scan_step	TEXT		The stage of the scan the line belongs to, for example Testing suspects.
log_message	TEXT		The text of the line.

Table 45. ScanLogEntry table

Relationships. Each line below reads from the first table to the second.

From	To	Meaning	Notes
Project	TestSetup	one Project has zero or many TestSetup	Each TestSetup belongs to exactly one Project
Project	Scan	one Project has zero or many Scan	Each Scan belongs to exactly one Project
TestSetup	TestAccount	one TestSetup has one or many TestAccount	Each TestAccount belongs to exactly one TestSetup

From	To	Meaning	Notes
TestSetup	Scan	one TestSetup has zero or many Scan	Each Scan belongs to zero or one TestSetup
Scan	Route	one Scan has zero or many Route	Each Route belongs to exactly one Scan
Scan	Finding	one Scan has zero or many Finding	Each Finding belongs to exactly one Scan
Route	Finding	one Route has zero or many Finding	Each Finding belongs to exactly one Route
Finding	Evidence	one Finding has zero or one Evidence	Each Evidence belongs to exactly one Finding
Finding	FixProposal	one Finding has zero or many FixProposal	Each FixProposal belongs to exactly one Finding
Scan	DatabaseCheck	one Scan has zero or one DatabaseCheck	Each DatabaseCheck belongs to exactly one Scan
Scan	Report	one Scan has zero or many Report	Each Report belongs to exactly one Scan
Scan	ScanLogEntry	one Scan has zero or many ScanLogEntry	Each ScanLogEntry belongs to exactly one Scan

Table 46. Relationships between the tables

Findings from different scans are matched through the finding key and not through a foreign key, because each scan stores its own findings.

Appendix C: To Be Determined List

No.	Open item	Planned closure
TBD-1	Destination of the "See supported apps" button on the No Flask app found screen (a short help page or a README section).	Design review, Semester 1
TBD-2	Optional AI-written summaries and fixes. Excluded from version 1.0; the project plan still lists them as an optional extra in Semester 2.	Decision with advisor; update the plan
TBD-3	Final values of the performance targets in Section 5.1 after measuring on the benchmark apps.	Testing, Semester 2
TBD-4	The list of 10 to 15 benchmark applications.	End of Semester 1
TBD-5	Exact minimum versions of VS Code, Docker and Python after testing on all three operating systems.	Testing, Semester 2
TBD-6	Whether to support PostgreSQL or MySQL after version 1.0.	Future work
TBD-7	Licence of the released code.	Before final submission
TBD-8	Final list of fix rules supported in version 1.0 (Section 4.9).	Design, Semester 1
TBD-9	How many scans to keep in the history before old ones are offered for deletion.	Design, Semester 1

Table 47. Open items